- Top 25 C++ Concurrency Interview Questions
  - 1. What is concurrency and how does it differ from parallelism?
    - Concurrency
    - Parallelism
    - Key Differences:
    - Detailed Examples:
      - Example 1: Web Server (Concurrency without Parallelism)
      - Example 2: Parallel Matrix Multiplication
      - Example 3: Producer-Consumer (Concurrent Coordination)
    - Real-World Analogies:
      - Concurrency: Single Chef, Multiple Dishes
      - Parallelism: Multiple Chefs, Multiple Dishes
    - Why the Distinction Matters:
    - Summary:
  - 2. What are the main threading primitives in C++11 and later?
    - 1. std::thread - Basic Thread Class
    - 2. std::mutex - Mutual Exclusion Primitive
    - 3. std::condition_variable - Synchronization Primitive
    - 4. std::atomic - Lock-Free Atomic Operations
    - 5. std::future/std::promise - Asynchronous Result Handling
    - Summary Table:
  - 3. Explain the difference between std::mutex and std::recursive_mutex.
    - std::mutex - Standard Mutex
    - std::recursive_mutex - Recursive Mutex
    - Key Differences:
    - Example 1: Problem with std::mutex
    - Example 2: Solution with std::recursive_mutex
    - Example 3: Recursive Function Scenario
    - Important Considerations:
    - When to Use Each:
  - 4. What is a race condition and how can you prevent it?
    - Example 1: Classic Race Condition
    - Why Race Conditions Happen:
    - Prevention Method 1: Using Mutexes
    - Prevention Method 2: Using Atomic Operations
    - Prevention Method 3: Thread-Safe Data Structures
    - Prevention Method 4: Proper Synchronization
    - Real-World Example: Bank Account

    - Prevention Methods Summary:
    - Best Practices:
- 5. What is the difference between std::lock_guard and std::unique_lock?
    - Comparison Table:
    - Example 1: Basic Usage - std::lock_guard
    - Example 2: Manual Lock/Unlock - std::unique_lock
    - Example 3: Condition Variable - Requires std::unique_lock
    - Example 4: Deferred Locking
    - Example 5: Try Lock
    - Example 6: Timed Lock
    - Example 7: Moving Locks
    - Example 8: Real-World Comparison
    - When to Use Each:
        - Use std::lock_guard when:
        - Use std::unique_lock when:
    - Performance Consideration:
    - C++17 Improvement: Class Template Argument Deduction (CTAD)
    - Summary:
- 6. Explain deadlock and strategies to avoid it.
    - Classic Deadlock Example (Dining Philosophers Problem)
    - Deadlock Conditions (All 4 must be present):
    - Strategy 1: Lock Ordering (Prevent Circular Wait)
    - Strategy 2: Using std::lock() (Atomic Multi-Lock)
    - Strategy 3: Using std::scoped_lock (C++17)
    - Strategy 4: Timeout-Based Locking
    - Strategy 5: Lock Hierarchy (Layered Locking)
    - Strategy 6: Avoid Nested Locks
    - Strategy 7: Use Lock-Free Data Structures
    - Deadlock Detection Example
    - Summary of Strategies:
    - Best Practices:
- 7. What are atomic operations and when should you use them?
    - Key Characteristics:
    - Basic Example: Atomic Counter
    - Common Atomic Operations
    - Use Case 1: Flags and Signals
    - Use Case 2: Spinlock Implementation
    - Use Case 3: Lock-Free Stack

    - Use Case 4: Reference Counting
    - Use Case 5: Progress Tracking
    - Memory Ordering Examples
    - When to Use Atomics:
    - Performance Comparison:
    - Summary:
- 8. Explain the memory ordering models in C++.
    - Memory Ordering Types:
    - 1. memory_order_relaxed - No Ordering
    - 2. memory_order_acquire and memory_order_release
    - 3. Acquire-Release Synchronization Pattern
    - 4. memory_order_acq_rel - Read-Modify-Write
    - 5. memory_order_seq_cst - Sequential Consistency (Default)
    - Real-World Example: Lock-Free Queue
    - Example: Double-Checked Locking Pattern
    - Memory Ordering Decision Tree:
    - Performance Impact:
    - Common Pitfalls:
    - Summary:
- 9. What is std::condition_variable and how do you use it?
    - Key Concepts:
    - Basic Example: Producer-Consumer
    - Why Predicates Are Important (Spurious Wakeups)
    - Thread-Safe Queue with Condition Variable
    - Multiple Producers, Multiple Consumers
    - Event Synchronization
    - Barrier Implementation
    - notify_one() vs notify_all()
    - Best Practices:
    - Common Pitfalls:
- 10. What is the difference between std::async and std::thread?
    - Comparison Table:
    - Example 1: Basic Usage
    - Example 2: Return Values
    - Example 3: Exception Handling
    - Example 4: Launch Policies
    - Example 5: Resource Management
    - Example 6: Parallel Algorithm Pattern

    - Example 7: When to Use Each
    - Example 8: Thread Pool vs Async
    - Future Status Checking
    - When to Use Which:
        - Use std::thread when:
        - Use std::async when:
    - Performance Considerations:
    - Summary:
- 11. Explain the producer-consumer problem and its solution.
    - Problem Challenges:
    - Solution 1: Using Mutex and Condition Variables
    - Solution 2: Using Semaphores (C++20)
    - Solution 3: Lock-Free Ring Buffer (Single Producer, Single Consumer)
    - Solution 4: Using std::queue with Timeout
    - Real-World Example: Task Queue
    - Key Design Decisions:
    - Performance Considerations:
    - Summary:
- 12. What is a thread pool and why is it useful?
    - Benefits:
    - Basic Thread Pool Implementation
    - Advanced Thread Pool with Return Values
    - Thread Pool with Priority Queue
    - Dynamic Thread Pool (Scales Up/Down)
    - Real-World Example: Web Server Request Handler
    - Performance Comparison
    - When to Use Thread Pools:
    - Key Design Considerations:
    - Summary:
- 13. What are the different types of mutexes in C++?
    - 1. std::mutex - Basic Mutex
    - 2. std::recursive_mutex - Recursive Mutex
    - 3. std::timed_mutex - Mutex with Timeout
    - 4. std::recursive_timed_mutex - Recursive + Timed
    - 5. std::shared_mutex (C++17) - Reader-Writer Lock
    - 6. std::shared_timed_mutex (C++14) - Shared + Timed
    - Comparison Table:
    - Real-World Example: Configuration Manager

    - Decision Tree:
    - Performance Impact (Approximate):
    - Summary:
- 14. Explain std::shared_mutex and reader-writer locks.
    - Basic Concept:
    - Basic Example:
    - Lock Types:
    - Advanced Example: Caching System
    - Upgrade Lock Pattern (Read → Write)
    - Performance Comparison:
    - Read-Write Ratio Matters:
    - Fairness and Starvation:
    - Best Practices:
    - When to Use:
    - Summary:
- 15. What is lock-free programming?
- 16. What is the ABA problem in concurrent programming?
- 17. Explain std::call_once and std::once_flag.
- 18. What is the difference between join() and detach() for threads?
- 19. What are futures and promises in C++?
- 20. Explain thread-local storage.
- 21. What is spurious wakeup in condition variables?
- 22. How do you implement a thread-safe singleton?
- 23. What is the difference between cooperative and preemptive multitasking?
- 24. Explain memory barriers and their importance.
- 25. What are the common pitfalls in concurrent programming?
- Additional Tips for Interviews:
- Code Examples to Practice:

Top 25 C++ Concurrency Interview Questions

---

1. What is concurrency and how does it differ from parallelism?

Answer:

Concurrency

Concurrency is about **dealing with** multiple tasks at once. It's about the **structure** and **composition** of your program - how you organize and manage multiple independent activities. Concurrent programs can run on single-core systems through time-slicing (interleaving execution).

Parallelism

Parallelism is about **doing** multiple tasks simultaneously. It's about actual **execution** - physically running multiple tasks at the exact same time on multiple cores/processors.

Key Differences:

Aspect	Concurrency	Parallelism
Focus	Program structure/composition	Actual execution
Hardware	Can work on single core	Requires multiple cores
Goal	Managing complexity	Improving performance
Implementation	Task scheduling, coordination	Task distribution

Detailed Examples:

Example 1: Web Server (Concurrency without Parallelism)

```
// Single-threaded concurrent web server using async I/O
class WebServer {
    std::queue<Request> requests;

public:
    void handleRequests() {
        while (true) {
            // Check for new connections (non-blocking)
            if (auto newReq = checkForNewConnection()) {
                requests.push(*newReq);
            }

            // Process existing requests (time-sliced)
            if (!requests.empty()) {
                auto req = requests.front();
```

```
                requests.pop();

                if (req.isComplete()) {
                    sendResponse(req);
                } else {
                    continueProcessing(req);
                    requests.push(req); // Put back if not done
                }
            }
        }
    }
};
```

This is concurrent but not parallel - one thread handles multiple requests by switching between them.

Example 2: Parallel Matrix Multiplication

```
#include <thread>
#include <vector>

void multiplyBlock(const Matrix& A, const Matrix& B, Matrix& C,
                   int startRow, int endRow) {
    for (int i = startRow; i < endRow; ++i) {
        for (int j = 0; j < B.cols(); ++j) {
            for (int k = 0; k < A.cols(); ++k) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void parallelMatrixMultiply(const Matrix& A, const Matrix& B, Matrix& C) {
    int numThreads = std::thread::hardware_concurrency();
    std::vector<std::thread> threads;

    int rowsPerThread = A.rows() / numThreads;

    // Create threads (parallel execution)
    for (int i = 0; i < numThreads; ++i) {
        int startRow = i * rowsPerThread;
        int endRow = (i == numThreads - 1) ? A.rows() : (i + 1) *
rowsPerThread;

        threads.emplace_back(multiplyBlock, std::cref(A), std::cref(B),
                             std::ref(C), startRow, endRow);
    }

    // Wait for all threads
    for (auto& t : threads) {
        t.join();
    }
}
```

This is both concurrent and parallel - multiple threads actually run simultaneously on different cores.

Example 3: Producer-Consumer (Concurrent Coordination)

```
#include <queue>
#include <mutex>
#include <condition_variable>
#include <thread>

template<typename T>
class ThreadSafeQueue {
private:
    mutable std::mutex mtx;
    std::queue<T> queue;
    std::condition_variable cv;

public:
    void push(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        queue.push(value);
        cv.notify_one();
    }

    bool pop(T& value) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this] { return !queue.empty(); });

        value = queue.front();
        queue.pop();
        return true;
    }
};

// Producer thread
void producer(ThreadSafeQueue<int>& q) {
    for (int i = 0; i < 100; ++i) {
        q.push(i);
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

// Consumer thread
void consumer(ThreadSafeQueue<int>& q) {
    int value;
    while (q.pop(value)) {
        std::cout << "Consumed: " << value << std::endl;
        // Process the value...
    }
}
```

This demonstrates concurrent coordination - threads cooperate through synchronization primitives.

Real-World Analogies:

Concurrency: Single Chef, Multiple Dishes

- One chef (single core) cooking multiple dishes
- Chef switches between tasks: chop vegetables → check oven → stir sauce → back to chopping
- All dishes progress, but only one task happens at a time
- **Structure**: How the chef organizes and manages multiple cooking tasks

Parallelism: Multiple Chefs, Multiple Dishes

- Multiple chefs (multiple cores) working simultaneously
- Each chef works on different dishes at the same time
- All tasks actually happen simultaneously
- **Execution**: Multiple chefs physically working at the same moment

Why the Distinction Matters:

1. **Design Implications**:
    - Concurrent design focuses on coordination, synchronization, and avoiding race conditions
    - Parallel design focuses on workload distribution and load balancing
2. **Performance**:
    - Concurrency can improve responsiveness and resource utilization
    - Parallelism can improve throughput and computational speed
3. **Debugging**:
    - Concurrent bugs: race conditions, deadlocks, starvation
    - Parallel bugs: load imbalance, false sharing, synchronization overhead
4. **Scalability**:

- Concurrent programs can benefit from better I/O handling
- Parallel programs scale with number of cores

Summary:

- **Concurrency = Composition** (dealing with multiple things at once)
- **Parallelism = Execution** (doing multiple things at once)
- You can have concurrency without parallelism, but parallelism implies concurrency
- Both are essential tools for modern C++ programming

2. What are the main threading primitives in C++11 and later?

Answer:

C++11 introduced comprehensive threading support in the standard library. Here are the main primitives:

1. `std::thread` - Basic Thread Class

Creates and manages threads of execution.

```
#include <thread>
#include <iostream>

void printNumbers(int start, int end) {
    for (int i = start; i <= end; ++i) {
        std::cout << i << " ";
    }
}

int main() {
    std::thread t1(printNumbers, 1, 5);
    std::thread t2(printNumbers, 6, 10);

    t1.join();  // Wait for t1 to finish
    t2.join();  // Wait for t2 to finish

    return 0;
}
```

2. `std::mutex` - Mutual Exclusion Primitive

Protects shared data from concurrent access.

```
#include <mutex>
#include <thread>

std::mutex mtx;
int shared_counter = 0;

void incrementCounter(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        std::lock_guard<std::mutex> lock(mtx);  // RAII lock
        ++shared_counter;
    }
}

int main() {
    std::thread t1(incrementCounter, 1000);
    std::thread t2(incrementCounter, 1000);

    t1.join();
    t2.join();

    std::cout << "Counter: " << shared_counter << std::endl;  // 2000
    return 0;
}
```

3. `std::condition_variable` - Synchronization Primitive

Allows threads to wait for certain conditions to be met.

```
#include <condition_variable>
#include <mutex>
#include <queue>

std::queue<int> data_queue;
std::mutex mtx;
std::condition_variable cv;
bool done = false;

void producer() {
    for (int i = 0; i < 10; ++i) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            data_queue.push(i);
        }
```

```
        cv.notify_one();  // Notify waiting consumer
    }
    {
        std::lock_guard<std::mutex> lock(mtx);
        done = true;
    }
    cv.notify_all();
}

void consumer() {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [] { return !data_queue.empty() || done; });

        while (!data_queue.empty()) {
            int value = data_queue.front();
            data_queue.pop();
            lock.unlock();
            std::cout << "Consumed: " << value << std::endl;
            lock.lock();
        }

        if (done && data_queue.empty()) break;
    }
}
```

4. `std::atomic` - Lock-Free Atomic Operations

Provides atomic operations without locks for simple types.

```
#include <atomic>
#include <thread>
#include <vector>

std::atomic<int> atomic_counter(0);

void incrementAtomic(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        atomic_counter.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(incrementAtomic, 1000);
    }

    for (auto& t : threads) {
        t.join();
    }
```

```
    std::cout << "Atomic counter: " << atomic_counter << std::endl;  // 10000
    return 0;
}
```

5. `std::future/std::promise` - Asynchronous Result Handling

Provides a mechanism for transferring data between threads asynchronously.

```
#include <future>
#include <thread>
#include <iostream>

int computeValue(int x) {
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return x * x;
}

int main() {
    // Using std::async
    std::future<int> result1 = std::async(std::launch::async, computeValue,
5);
    std::cout << "Computing..." << std::endl;
    std::cout << "Result: " << result1.get() << std::endl;  // 25

    // Using std::promise
    std::promise<int> promise;
    std::future<int> result2 = promise.get_future();

    std::thread t([&promise] {
        std::this_thread::sleep_for(std::chrono::seconds(1));
        promise.set_value(42);
    });

    std::cout << "Waiting for promise..." << std::endl;
    std::cout << "Promise result: " << result2.get() << std::endl;  // 42

    t.join();
    return 0;
}
```

Summary Table:

Primitive	Use Case	Performance	Complexity
`std::thread`	Thread creation	N/A	Low

Primitive	Use Case	Performance	Complexity
`std::mutex`	Protecting shared data	Medium	Low
`std::condition_variable`	Thread coordination	Medium	Medium
`std::atomic`	Lock-free operations	High	Medium-High
`std::future/promise`	Async results	Medium	Low

3. Explain the difference between `std::mutex` and `std::recursive_mutex`.

Answer:

`std::mutex` - Standard Mutex

A standard mutex cannot be locked multiple times by the same thread. Attempting to do so results in **undefined behavior** (usually deadlock).

`std::recursive_mutex` - Recursive Mutex

A recursive mutex can be locked multiple times by the **same thread**. Each lock must be matched with a corresponding unlock.

Key Differences:

Feature	`std::mutex`	`std::recursive_mutex`
Multiple locks by same thread	❌ Undefined behavior	✅ Allowed
Lock count tracking	No	Yes
Performance	Faster	Slower (overhead)
Use case	General synchronization	Recursive functions

Example 1: Problem with `std::mutex`

```
#include <mutex>
#include <iostream>

class BankAccount {
    std::mutex mtx;
    double balance;

public:
    BankAccount(double initial) : balance(initial) {}

    void deposit(double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        balance += amount;
    }

    void transfer(BankAccount& other, double amount) {
        std::lock_guard<std::mutex> lock(mtx);  // Lock 1
        balance -= amount;
        other.deposit(amount);  // Tries to lock again - DEADLOCK!
    }

    double getBalance() {
        std::lock_guard<std::mutex> lock(mtx);
        return balance;
    }
};

// This will DEADLOCK on self-transfer
void problemExample() {
    BankAccount account(1000.0);
    account.transfer(account, 100.0);  // DEADLOCK!
}
```

Example 2: Solution with `std::recursive_mutex`

```
#include <mutex>
#include <iostream>

class BankAccount {
    std::recursive_mutex mtx;  // Changed to recursive_mutex
    double balance;

public:
    BankAccount(double initial) : balance(initial) {}

    void deposit(double amount) {
```

```
        std::lock_guard<std::recursive_mutex> lock(mtx);
        balance += amount;
        std::cout << "Deposited: " << amount
                  << ", Balance: " << balance << std::endl;
    }

    void transfer(BankAccount& other, double amount) {
        std::lock_guard<std::recursive_mutex> lock(mtx);  // Lock 1
        if (&other == this) {
            // Self-transfer - recursive lock happens here
            deposit(amount);  // Lock 2 (same thread) - OK!
            withdraw(amount); // Lock 3 (same thread) - OK!
        } else {
            balance -= amount;
            other.deposit(amount);
        }
    }

    void withdraw(double amount) {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        balance -= amount;
        std::cout << "Withdrew: " << amount
                  << ", Balance: " << balance << std::endl;
    }

    double getBalance() {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        return balance;
    }
};

int main() {
    BankAccount account(1000.0);
    account.transfer(account, 100.0);  // Works fine now!
    std::cout << "Final balance: " << account.getBalance() << std::endl;
    return 0;
}
```

Example 3: Recursive Function Scenario

```
#include <mutex>
#include <iostream>
#include <vector>

class FileSystem {
    std::recursive_mutex mtx;
    std::vector<std::string> files;

public:
    void addFile(const std::string& filename) {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        files.push_back(filename);
    }
```

```
    // Recursive function that needs locking
    void addFilesRecursively(const std::vector<std::string>& filenames,
                             size_t index = 0) {
        std::lock_guard<std::recursive_mutex> lock(mtx);  // Locks on each
call

        if (index >= filenames.size()) return;

        files.push_back(filenames[index]);
        std::cout << "Added: " << filenames[index] << std::endl;

        // Recursive call - locks again (same thread)
        addFilesRecursively(filenames, index + 1);
    }

    size_t getFileCount() {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        return files.size();
    }
};

int main() {
    FileSystem fs;
    std::vector<std::string> files = {"file1.txt", "file2.txt", "file3.txt"};

    fs.addFilesRecursively(files);
    std::cout << "Total files: " << fs.getFileCount() << std::endl;

    return 0;
}
```

Important Considerations:

1. **Performance**: `std::recursive_mutex` is slower due to lock count tracking
2. **Design Smell**: Need for recursive mutex often indicates poor design
3. **Better Alternative**: Refactor to avoid recursive locking when possible

```
// Better design - internal unlocked version
class BetterBankAccount {
    std::mutex mtx;
    double balance;

    // Internal version - assumes lock already held
    void depositInternal(double amount) {
        balance += amount;
    }

public:
    void deposit(double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        depositInternal(amount);  // No locking needed
```

```
    }

    void transfer(BetterBankAccount& other, double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        balance -= amount;
        depositInternal(amount);  // Safe - uses internal version
    }
};
```

When to Use Each:

- `std::mutex`: Default choice for most synchronization needs
- `std::recursive_mutex`: Only when you have legitimate recursive locking needs (rare)
- **Better approach**: Redesign to avoid recursive locking if possible

4. What is a race condition and how can you prevent it?

Answer:

A **race condition** occurs when multiple threads access shared data concurrently, and at least one thread modifies the data, leading to unpredictable and incorrect results. The outcome depends on the unpredictable timing of thread execution.

Example 1: Classic Race Condition

```
#include <thread>
#include <iostream>
#include <vector>

int counter = 0;  // Shared variable

void incrementCounter(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        // Race condition: Read-Modify-Write is not atomic
        counter++;  // Actually: temp = counter; temp++; counter = temp;
    }
}

int main() {
    std::vector<std::thread> threads;
```

```
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(incrementCounter, 10000);
    }

    for (auto& t : threads) {
        t.join();
    }

    // Expected: 100000, Actual: varies (e.g., 87432, 93421, etc.)
    std::cout << "Counter: " << counter << std::endl;
    return 0;
}
```

Why Race Conditions Happen:

```
Thread 1: Read counter (0) → Increment (1) → Write (1)
Thread 2:                  Read counter (0) → Increment (1) → Write (1)
Result: counter = 1 (should be 2!)
```

Prevention Method 1: Using Mutexes

```
#include <mutex>
#include <thread>
#include <vector>

int counter = 0;
std::mutex counter_mutex;

void safeIncrementCounter(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        std::lock_guard<std::mutex> lock(counter_mutex);
        counter++;  // Protected by mutex
    }
}

int main() {
    std::vector<std::thread> threads;

    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(safeIncrementCounter, 10000);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Counter: " << counter << std::endl;  // Always 100000
```

```
    return 0;
}
```

Prevention Method 2: Using Atomic Operations

```
#include <atomic>
#include <thread>
#include <vector>

std::atomic<int> atomic_counter(0);

void atomicIncrementCounter(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        atomic_counter.fetch_add(1, std::memory_order_relaxed);
        // Or simply: atomic_counter++;
    }
}

int main() {
    std::vector<std::thread> threads;

    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(atomicIncrementCounter, 10000);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Atomic counter: " << atomic_counter << std::endl;  // Always
100000
    return 0;
}
```

Prevention Method 3: Thread-Safe Data Structures

```
#include <mutex>
#include <queue>
#include <optional>

template<typename T>
class ThreadSafeQueue {
private:
    mutable std::mutex mtx;
    std::queue<T> queue;
```

```
public:
    void push(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        queue.push(std::move(value));
    }

    std::optional<T> pop() {
        std::lock_guard<std::mutex> lock(mtx);
        if (queue.empty()) {
            return std::nullopt;
        }
        T value = std::move(queue.front());
        queue.pop();
        return value;
    }

    bool empty() const {
        std::lock_guard<std::mutex> lock(mtx);
        return queue.empty();
    }

    size_t size() const {
        std::lock_guard<std::mutex> lock(mtx);
        return queue.size();
    }
};

// Usage
void producer(ThreadSafeQueue<int>& q) {
    for (int i = 0; i < 100; ++i) {
        q.push(i);
    }
}

void consumer(ThreadSafeQueue<int>& q) {
    while (true) {
        auto item = q.pop();
        if (item) {
            std::cout << "Consumed: " << *item << std::endl;
        } else {
            break;
        }
    }
}
```

Prevention Method 4: Proper Synchronization

```
#include <mutex>
#include <condition_variable>
#include <thread>

class Coordinator {
    std::mutex mtx;
```

```
    std::condition_variable cv;
    bool ready = false;
    int data = 0;

public:
    void producer() {
        std::this_thread::sleep_for(std::chrono::seconds(1));

        {
            std::lock_guard<std::mutex> lock(mtx);
            data = 42;  // Produce data
            ready = true;
        }

        cv.notify_one();  // Signal consumer
    }

    void consumer() {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this] { return ready; });  // Wait for data

        std::cout << "Consumed: " << data << std::endl;  // No race
    }
};
```

Real-World Example: Bank Account

```
#include <mutex>
#include <thread>

class BankAccount {
    mutable std::mutex mtx;
    double balance;

public:
    BankAccount(double initial) : balance(initial) {}

    // Thread-safe deposit
    void deposit(double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        balance += amount;
    }

    // Thread-safe withdrawal
    bool withdraw(double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        if (balance >= amount) {
            balance -= amount;
            return true;
        }
        return false;
    }
```

```
    // Thread-safe transfer (requires multiple locks)
    static bool transfer(BankAccount& from, BankAccount& to, double amount) {
        // Prevent deadlock by locking in consistent order
        std::lock(from.mtx, to.mtx);
        std::lock_guard<std::mutex> lock1(from.mtx, std::adopt_lock);
        std::lock_guard<std::mutex> lock2(to.mtx, std::adopt_lock);

        if (from.balance >= amount) {
            from.balance -= amount;
            to.balance += amount;
            return true;
        }
        return false;
    }

    double getBalance() const {
        std::lock_guard<std::mutex> lock(mtx);
        return balance;
    }
};
```

Prevention Methods Summary:

1. **Mutexes/Locks**: Protect critical sections
    - `std::mutex`, `std::lock_guard`, `std::unique_lock`
2. **Atomic Operations**: Lock-free for simple operations
    - `std::atomic<T>`
3. **Thread-Safe Data Structures**: Encapsulate synchronization
    - Custom thread-safe containers
4. **Proper Synchronization**: Coordinate thread execution
    - `std::condition_variable`
5. **Immutability**: Avoid shared mutable state
    - Pass by value, use const
6. **Thread-Local Storage**: Per-thread data
    - `thread_local` variables

Best Practices:

- **Minimize shared state**: Less sharing = fewer races
- **Use RAII locks**: Automatic unlock prevents errors
- **Prefer atomics** for simple counters/flags
- **Design for thread safety** from the start
- **Test thoroughly**: Race conditions are hard to reproduce

5. What is the difference between `std::lock_guard` and `std::unique_lock`?

Answer:

Both are RAII wrappers for mutexes, but `std::unique_lock` offers more flexibility at the cost of slightly more overhead.

Comparison Table:

Feature	`std::lock_guard`	`std::unique_lock`
Manual lock/unlock	❌ No	✅ Yes
Moveable	❌ No	✅ Yes
Works with `std::condition_variable`	❌ No	✅ Yes
Deferred locking	❌ No	✅ Yes
Timed locking	❌ No	✅ Yes
Try lock	❌ No	✅ Yes
Performance	Faster (minimal overhead)	Slightly slower
Use case	Simple critical sections	Complex scenarios

Example 1: Basic Usage - `std::lock_guard`

```cpp
#include <mutex>
#include <thread>
#include <iostream>

std::mutex mtx;
int shared_data = 0;

void simpleUpdate() {
    std::lock_guard<std::mutex> lock(mtx);  // Locks immediately
    shared_data++;
    std::cout << "Updated: " << shared_data << std::endl;
    // Automatically unlocks when lock goes out of scope
}

// Simple, efficient, but inflexible
```

Example 2: Manual Lock/Unlock - `std::unique_lock`

```cpp
#include <mutex>
#include <thread>
#include <iostream>

std::mutex mtx;

void complexUpdate() {
    std::unique_lock<std::mutex> lock(mtx);

    // Do some locked work
    std::cout << "Locked work" << std::endl;

    lock.unlock();  // Manually unlock

    // Do some work without lock
    std::cout << "Unlocked work" << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    lock.lock();  // Lock again

    // More locked work
    std::cout << "Locked again" << std::endl;

    // Automatically unlocks at scope exit
}
```

Example 3: Condition Variable - Requires `std::unique_lock`

```cpp
#include <mutex>
#include <condition_variable>
#include <queue>
#include <thread>

std::mutex mtx;
std::condition_variable cv;
std::queue<int> queue;
bool finished = false;

void producer() {
    for (int i = 0; i < 10; ++i) {
        {
            std::lock_guard<std::mutex> lock(mtx);  // Can use lock_guard here
            queue.push(i);
            std::cout << "Produced: " << i << std::endl;
        }
        cv.notify_one();
    }

    {
        std::lock_guard<std::mutex> lock(mtx);
        finished = true;
    }
    cv.notify_all();
}

void consumer() {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx);  // Must use unique_lock!

        // cv.wait() needs to unlock and relock
        cv.wait(lock, [] { return !queue.empty() || finished; });

        while (!queue.empty()) {
            int value = queue.front();
            queue.pop();
            std::cout << "Consumed: " << value << std::endl;
        }

        if (finished && queue.empty()) {
            break;
        }
    }
}
```

Example 4: Deferred Locking

```cpp
#include <mutex>
#include <thread>

std::mutex mtx1, mtx2;
```

```
void transferData() {
    // Create locks without locking immediately
    std::unique_lock<std::mutex> lock1(mtx1, std::defer_lock);
    std::unique_lock<std::mutex> lock2(mtx2, std::defer_lock);

    // Lock both atomically (prevents deadlock)
    std::lock(lock1, lock2);

    // Both locks are now owned
    // Critical section here

    // Both unlock automatically at scope exit
}

// Cannot do this with lock_guard!
```

Example 5: Try Lock

```
#include <mutex>
#include <thread>
#include <iostream>

std::mutex mtx;

void tryLockExample() {
    std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);

    if (lock.owns_lock()) {
        std::cout << "Lock acquired, doing work" << std::endl;
        // Do work
    } else {
        std::cout << "Could not acquire lock, doing alternative" << std::endl;
        // Do alternative work
    }
}
```

Example 6: Timed Lock

```
#include <mutex>
#include <chrono>
#include <thread>
#include <iostream>

std::timed_mutex tmtx;

void timedLockExample() {
    std::unique_lock<std::timed_mutex> lock(tmtx, std::defer_lock);
```

```
    if (lock.try_lock_for(std::chrono::milliseconds(100))) {
        std::cout << "Lock acquired within timeout" << std::endl;
        // Do work
    } else {
        std::cout << "Timeout! Could not acquire lock" << std::endl;
        // Handle timeout
    }
}
```

Example 7: Moving Locks

```
#include <mutex>
#include <thread>
#include <utility>

std::mutex mtx;

std::unique_lock<std::mutex> getLock() {
    std::unique_lock<std::mutex> lock(mtx);
    // Do some work
    return lock;  // Move the lock ownership
}

void moveLockExample() {
    std::unique_lock<std::mutex> my_lock = getLock();
    // my_lock now owns the mutex
    // Do more work
    // Unlocks automatically
}

// Cannot return lock_guard (not moveable)
```

Example 8: Real-World Comparison

```
#include <mutex>
#include <vector>
#include <thread>

class DataProcessor {
    std::mutex mtx;
    std::vector<int> data;

public:
    // Simple case: use lock_guard
    void addData(int value) {
        std::lock_guard<std::mutex> lock(mtx);
        data.push_back(value);
```

```
    }

    // Complex case: use unique_lock
    void processData() {
        std::unique_lock<std::mutex> lock(mtx);

        if (data.empty()) {
            return;
        }

        // Copy data while locked
        std::vector<int> local_copy = data;

        // Unlock before expensive processing
        lock.unlock();

        // Process without holding lock
        for (auto& value : local_copy) {
            value *= 2;  // Expensive operation
            std::this_thread::sleep_for(std::chrono::milliseconds(1));
        }

        // Lock again to update
        lock.lock();
        data = std::move(local_copy);
    }

    size_t size() const {
        std::lock_guard<std::mutex> lock(mtx);
        return data.size();
    }
};
```

When to Use Each:

Use `std::lock_guard` when:

- Simple lock/unlock pattern
- Lock held for entire scope
- Maximum performance needed
- No condition variables involved

Use `std::unique_lock` when:

- Need to manually lock/unlock
- Working with condition variables
- Need deferred locking
- Need to try lock or timed lock

- Need to transfer lock ownership
- Complex locking scenarios

Performance Consideration:

```
// lock_guard: ~2-3 CPU cycles overhead
std::lock_guard<std::mutex> lg(mtx);

// unique_lock: ~5-10 CPU cycles overhead (due to additional state tracking)
std::unique_lock<std::mutex> ul(mtx);
```

C++17 Improvement: Class Template Argument Deduction (CTAD)

```
std::mutex mtx;

// C++17: Can omit template parameter
std::lock_guard lock(mtx);      // Deduces std::lock_guard<std::mutex>
std::unique_lock ulock(mtx);    // Deduces std::unique_lock<std::mutex>
```

Summary:

- **Default choice**: Use `std::lock_guard` for simplicity and performance
- **Flexibility needed**: Use `std::unique_lock` for advanced scenarios
- Both provide RAII guarantees for exception safety

6. Explain deadlock and strategies to avoid it.

Answer:

A **deadlock** occurs when two or more threads are blocked forever, each waiting for the other to release a resource. No thread can proceed, and the program hangs.

Classic Deadlock Example (Dining Philosophers Problem)

```
#include <mutex>
#include <thread>
#include <iostream>

std::mutex fork1, fork2;

void philosopher1() {
    std::lock_guard<std::mutex> lock1(fork1);  // Lock fork1
    std::cout << "Philosopher 1 got fork1, waiting for fork2..." << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    std::lock_guard<std::mutex> lock2(fork2);  // Wait for fork2 - DEADLOCK!
    std::cout << "Philosopher 1 eating" << std::endl;
}

void philosopher2() {
    std::lock_guard<std::mutex> lock1(fork2);  // Lock fork2
    std::cout << "Philosopher 2 got fork2, waiting for fork1..." << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    std::lock_guard<std::mutex> lock2(fork1);  // Wait for fork1 - DEADLOCK!
    std::cout << "Philosopher 2 eating" << std::endl;
}

// This will DEADLOCK
void deadlockExample() {
    std::thread t1(philosopher1);
    std::thread t2(philosopher2);
    t1.join();
    t2.join();
}
```

Deadlock Conditions (All 4 must be present):

1. **Mutual Exclusion**: Resources cannot be shared
2. **Hold and Wait**: Thread holds resources while waiting for others
3. **No Preemption**: Resources cannot be forcibly taken
4. **Circular Wait**: Circular chain of threads waiting for resources

Strategy 1: Lock Ordering (Prevent Circular Wait)

```cpp
#include <mutex>
#include <thread>
#include <iostream>

std::mutex fork1, fork2;

void philosopher1Safe() {
    // Always lock in same order: fork1, then fork2
    std::lock_guard<std::mutex> lock1(fork1);
    std::cout << "Philosopher 1 got fork1" << std::endl;

    std::lock_guard<std::mutex> lock2(fork2);
    std::cout << "Philosopher 1 got fork2, eating" << std::endl;
}

void philosopher2Safe() {
    // Same order: fork1, then fork2 (not fork2, then fork1)
    std::lock_guard<std::mutex> lock1(fork1);
    std::cout << "Philosopher 2 got fork1" << std::endl;

    std::lock_guard<std::mutex> lock2(fork2);
    std::cout << "Philosopher 2 got fork2, eating" << std::endl;
}

// No deadlock - consistent ordering
void safeExample() {
    std::thread t1(philosopher1Safe);
    std::thread t2(philosopher2Safe);
    t1.join();
    t2.join();
}
```

Strategy 2: Using `std::lock()` (Atomic Multi-Lock)

```cpp
#include <mutex>
#include <thread>
#include <iostream>

class BankAccount {
    std::mutex mtx;
    int balance;
    int id;

public:
    BankAccount(int id, int initial) : id(id), balance(initial) {}

    int getId() const { return id; }

    // Deadlock-free transfer using std::lock
    static void transfer(BankAccount& from, BankAccount& to, int amount) {
```

```
        // Lock both mutexes atomically - no deadlock possible!
        std::lock(from.mtx, to.mtx);

        // Adopt the already-locked mutexes
        std::lock_guard<std::mutex> lock1(from.mtx, std::adopt_lock);
        std::lock_guard<std::mutex> lock2(to.mtx, std::adopt_lock);

        from.balance -= amount;
        to.balance += amount;

        std::cout << "Transferred " << amount
                  << " from account " << from.id
                  << " to account " << to.id << std::endl;
    }

    int getBalance() {
        std::lock_guard<std::mutex> lock(mtx);
        return balance;
    }
};

void transferTest() {
    BankAccount acc1(1, 1000);
    BankAccount acc2(2, 1000);

    std::thread t1([&] { BankAccount::transfer(acc1, acc2, 100); });
    std::thread t2([&] { BankAccount::transfer(acc2, acc1, 50); });

    t1.join();
    t2.join();

    std::cout << "Account 1: " << acc1.getBalance() << std::endl;
    std::cout << "Account 2: " << acc2.getBalance() << std::endl;
}
```

Strategy 3: Using `std::scoped_lock` (C++17)

```
#include <mutex>
#include <thread>

std::mutex mtx1, mtx2, mtx3;

void multiLockSafe() {
    // C++17: Locks all mutexes atomically, no deadlock
    std::scoped_lock lock(mtx1, mtx2, mtx3);

    // All mutexes are locked
    // Critical section

    // All unlock automatically
}

// Equivalent to std::lock + multiple lock_guards, but simpler
```

Strategy 4: Timeout-Based Locking

```
#include <mutex>
#include <chrono>
#include <thread>
#include <iostream>

std::timed_mutex tmtx1, tmtx2;

bool tryTransferWithTimeout() {
    std::unique_lock<std::timed_mutex> lock1(tmtx1, std::defer_lock);
    std::unique_lock<std::timed_mutex> lock2(tmtx2, std::defer_lock);

    // Try to lock both with timeout
    auto timeout = std::chrono::milliseconds(100);

    if (lock1.try_lock_for(timeout)) {
        if (lock2.try_lock_for(timeout)) {
            // Both locks acquired
            std::cout << "Transfer successful" << std::endl;
            return true;
        } else {
            std::cout << "Could not acquire second lock, aborting" <<
std::endl;
            return false;
        }
    } else {
        std::cout << "Could not acquire first lock, aborting" << std::endl;
        return false;
    }
}
```

Strategy 5: Lock Hierarchy (Layered Locking)

```
#include <mutex>
#include <thread>
#include <stdexcept>

class HierarchicalMutex {
    std::mutex internal_mutex;
    unsigned long const hierarchy_value;
    unsigned long previous_hierarchy_value;

    static thread_local unsigned long this_thread_hierarchy_value;

    void check_for_hierarchy_violation() {
        if (this_thread_hierarchy_value <= hierarchy_value) {
            throw std::logic_error("Mutex hierarchy violated");
```

```
        }
    }

    void update_hierarchy_value() {
        previous_hierarchy_value = this_thread_hierarchy_value;
        this_thread_hierarchy_value = hierarchy_value;
    }

public:
    explicit HierarchicalMutex(unsigned long value)
        : hierarchy_value(value),
          previous_hierarchy_value(0) {}

    void lock() {
        check_for_hierarchy_violation();
        internal_mutex.lock();
        update_hierarchy_value();
    }

    void unlock() {
        this_thread_hierarchy_value = previous_hierarchy_value;
        internal_mutex.unlock();
    }

    bool try_lock() {
        check_for_hierarchy_violation();
        if (!internal_mutex.try_lock()) {
            return false;
        }
        update_hierarchy_value();
        return true;
    }
};

thread_local unsigned long
HierarchicalMutex::this_thread_hierarchy_value(ULONG_MAX);

// Usage: Define hierarchy levels
HierarchicalMutex high_level_mutex(10000);
HierarchicalMutex mid_level_mutex(5000);
HierarchicalMutex low_level_mutex(1000);

void hierarchicalExample() {
    std::lock_guard<HierarchicalMutex> lock1(high_level_mutex);  // OK
    std::lock_guard<HierarchicalMutex> lock2(mid_level_mutex);   // OK
    std::lock_guard<HierarchicalMutex> lock3(low_level_mutex);   // OK

    // Trying to lock high_level_mutex again would throw exception
}
```

Strategy 6: Avoid Nested Locks

```
#include <mutex>
#include <vector>

class DataStore {
    std::mutex mtx;
    std::vector<int> data;

public:
    // Bad: Nested locking potential
    void badUpdate(DataStore& other) {
        std::lock_guard<std::mutex> lock1(mtx);
        // What if other.addData() also locks? Potential deadlock!
        other.addData(42);
    }

    // Good: Avoid nested locks
    void goodUpdate(int value) {
        std::lock_guard<std::mutex> lock(mtx);
        data.push_back(value);
    }

private:
    void addData(int value) {
        std::lock_guard<std::mutex> lock(mtx);
        data.push_back(value);
    }
};
```

Strategy 7: Use Lock-Free Data Structures

```
#include <atomic>
#include <memory>

template<typename T>
class LockFreeStack {
    struct Node {
        T data;
        Node* next;
        Node(const T& data_) : data(data_), next(nullptr) {}
    };

    std::atomic<Node*> head;

public:
    LockFreeStack() : head(nullptr) {}

    void push(const T& data) {
        Node* new_node = new Node(data);
        new_node->next = head.load();
        while (!head.compare_exchange_weak(new_node->next, new_node));
    }
```

```
    // No locks = no deadlocks!
};
```

Deadlock Detection Example

```
#include <mutex>
#include <thread>
#include <chrono>
#include <iostream>

class DeadlockDetector {
    std::timed_mutex mtx;
    static constexpr auto TIMEOUT = std::chrono::seconds(5);

public:
    bool safeOperation() {
        if (mtx.try_lock_for(TIMEOUT)) {
            std::lock_guard<std::timed_mutex> lock(mtx, std::adopt_lock);
            // Do work
            return true;
        } else {
            std::cerr << "Potential deadlock detected! Timeout exceeded." <<
std::endl;
            return false;
        }
    }
};
```

Summary of Strategies:

1. **Lock Ordering**: Always acquire locks in the same order
2. `std::lock()`: Atomically lock multiple mutexes
3. `std::scoped_lock` **(C++17)**: RAII version of `std::lock()`
4. **Timeout-Based Locking**: Give up if lock not acquired in time
5. **Lock Hierarchy**: Enforce ordering through hierarchy levels
6. **Avoid Nested Locks**: Design to minimize nested locking
7. **Lock-Free Structures**: Use atomics instead of locks
8. **Deadlock Detection**: Monitor and detect potential deadlocks

Best Practices:

- **Keep critical sections short**: Less time locked = less deadlock chance

- **Use RAII**: `lock_guard`, `unique_lock`, `scoped_lock`
- **Prefer higher-level abstractions**: Use `std::async`, thread pools
- **Test thoroughly**: Deadlocks may be timing-dependent
- **Document locking order**: Make requirements explicit

7. What are atomic operations and when should you use them?

Answer:

Atomic operations are indivisible operations that complete without interruption from other threads. They provide thread-safe access to shared variables without using locks.

Key Characteristics:

- **Indivisible**: Operation completes as a single unit
- **Thread-safe**: No data races
- **Lock-free**: No mutex overhead
- **Memory ordering**: Provides synchronization guarantees

Basic Example: Atomic Counter

```
#include <atomic>
#include <thread>
#include <vector>
#include <iostream>

// Non-atomic (UNSAFE)
int regular_counter = 0;

// Atomic (SAFE)
std::atomic<int> atomic_counter(0);

void incrementRegular(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        regular_counter++;  // NOT atomic: read, increment, write
    }
}

void incrementAtomic(int iterations) {
    for (int i = 0; i < iterations; ++i) {
        atomic_counter++;  // Atomic operation
```

```
        // Or: atomic_counter.fetch_add(1);
    }
}

int main() {
    const int num_threads = 10;
    const int iterations = 10000;

    // Test regular counter
    {
        std::vector<std::thread> threads;
        for (int i = 0; i < num_threads; ++i) {
            threads.emplace_back(incrementRegular, iterations);
        }
        for (auto& t : threads) t.join();

        std::cout << "Regular counter: " << regular_counter
                  << " (expected: " << num_threads * iterations << ")" <<
std::endl;
        // Output: varies, incorrect (e.g., 87234)
    }

    // Test atomic counter
    {
        std::vector<std::thread> threads;
        for (int i = 0; i < num_threads; ++i) {
            threads.emplace_back(incrementAtomic, iterations);
        }
        for (auto& t : threads) t.join();

        std::cout << "Atomic counter: " << atomic_counter
                  << " (expected: " << num_threads * iterations << ")" <<
std::endl;
        // Output: always 100000 (correct)
    }

    return 0;
}
```

Common Atomic Operations

```
#include <atomic>
#include <iostream>

void atomicOperationsDemo() {
    std::atomic<int> value(0);

    // Store
    value.store(42);

    // Load
    int current = value.load();
```

```
    // Exchange (set new value, return old)
    int old = value.exchange(100);

    // Fetch and add
    int previous = value.fetch_add(5);  // value becomes 105

    // Fetch and subtract
    previous = value.fetch_sub(10);  // value becomes 95

    // Compare and exchange (CAS)
    int expected = 95;
    bool success = value.compare_exchange_strong(expected, 200);
    if (success) {
        std::cout << "Value changed to 200" << std::endl;
    } else {
        std::cout << "Value was " << expected << ", not 95" << std::endl;
    }

    // Atomic operations with operators
    value++;  // Same as value.fetch_add(1)
    ++value;
    value--;
    --value;
    value += 10;
    value -= 5;
}
```

Use Case 1: Flags and Signals

```
#include <atomic>
#include <thread>
#include <iostream>

class Worker {
    std::atomic<bool> stop_flag{false};
    std::atomic<bool> ready{false};

public:
    void doWork() {
        // Signal that we're ready
        ready.store(true, std::memory_order_release);

        while (!stop_flag.load(std::memory_order_acquire)) {
            // Do work
            std::cout << "Working..." << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }

        std::cout << "Stopped" << std::endl;
    }

    void stop() {
        stop_flag.store(true, std::memory_order_release);
```

```
    }

    bool isReady() const {
        return ready.load(std::memory_order_acquire);
    }
};

void flagExample() {
    Worker worker;

    std::thread t(&Worker::doWork, &worker);

    // Wait for worker to be ready
    while (!worker.isReady()) {
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }

    std::this_thread::sleep_for(std::chrono::seconds(1));
    worker.stop();

    t.join();
}
```

Use Case 2: Spinlock Implementation

```
#include <atomic>
#include <thread>

class Spinlock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;

public:
    void lock() {
        // Spin until we acquire the lock
        while (flag.test_and_set(std::memory_order_acquire)) {
            // Busy wait
        }
    }

    void unlock() {
        flag.clear(std::memory_order_release);
    }
};

// Usage
Spinlock spinlock;

void criticalSection() {
    spinlock.lock();
    // Critical section - protected data access
    spinlock.unlock();
}
```

Use Case 3: Lock-Free Stack

```
#include <atomic>
#include <memory>

template<typename T>
class LockFreeStack {
private:
    struct Node {
        T data;
        Node* next;
        Node(const T& data_) : data(data_), next(nullptr) {}
    };

    std::atomic<Node*> head;

public:
    LockFreeStack() : head(nullptr) {}

    void push(const T& data) {
        Node* new_node = new Node(data);
        new_node->next = head.load(std::memory_order_relaxed);

        // CAS loop: retry until successful
        while (!head.compare_exchange_weak(new_node->next, new_node,
                                           std::memory_order_release,
                                           std::memory_order_relaxed)) {
            // Loop continues if CAS fails
        }
    }

    bool pop(T& result) {
        Node* old_head = head.load(std::memory_order_relaxed);

        while (old_head &&
               !head.compare_exchange_weak(old_head, old_head->next,
                                           std::memory_order_acquire,
                                           std::memory_order_relaxed)) {
            // Loop continues if CAS fails
        }

        if (old_head) {
            result = old_head->data;
            delete old_head;
            return true;
        }
        return false;
    }
};
```

Use Case 4: Reference Counting

```
#include <atomic>
#include <iostream>

class RefCounted {
    mutable std::atomic<int> ref_count{0};

public:
    void addRef() const {
        ref_count.fetch_add(1, std::memory_order_relaxed);
    }

    void release() const {
        if (ref_count.fetch_sub(1, std::memory_order_acq_rel) == 1) {
            delete this;
        }
    }

    int getRefCount() const {
        return ref_count.load(std::memory_order_relaxed);
    }

protected:
    RefCounted() = default;
    virtual ~RefCounted() {
        std::cout << "RefCounted destroyed" << std::endl;
    }
};
```

Use Case 5: Progress Tracking

```
#include <atomic>
#include <thread>
#include <vector>
#include <iostream>

class ProgressTracker {
    std::atomic<size_t> completed_tasks{0};
    size_t total_tasks;

public:
    ProgressTracker(size_t total) : total_tasks(total) {}

    void taskCompleted() {
        size_t current = completed_tasks.fetch_add(1,
std::memory_order_relaxed) + 1;
        if (current % 100 == 0) {
            std::cout << "Progress: " << current << "/" << total_tasks <<
std::endl;
        }
    }

    double getProgress() const {
```

```
        return static_cast<double>
(completed_tasks.load(std::memory_order_relaxed))
               / total_tasks * 100.0;
    }

    bool isComplete() const {
        return completed_tasks.load(std::memory_order_relaxed) >= total_tasks;
    }
};

void worker(ProgressTracker& tracker) {
    for (int i = 0; i < 100; ++i) {
        // Do work
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
        tracker.taskCompleted();
    }
}
```

Memory Ordering Examples

```
#include <atomic>
#include <thread>
#include <cassert>

// Relaxed ordering (no synchronization)
std::atomic<int> x{0}, y{0};

void relaxedExample() {
    x.store(1, std::memory_order_relaxed);
    y.store(1, std::memory_order_relaxed);
    // No ordering guarantees between stores
}

// Acquire-Release ordering
std::atomic<bool> ready{false};
int data = 0;

void producer() {
    data = 42;  // Non-atomic write
    ready.store(true, std::memory_order_release);  // Release: publishes all
previous writes
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) {  // Acquire: sees all
previous writes
        // Wait
    }
    assert(data == 42);  // Guaranteed to be true
}
```

When to Use Atomics:

✅ Use atomics when:

- Simple counters, flags, or indices
- Lock-free data structures
- Performance-critical code
- Progress indicators
- Reference counting
- Simple producer-consumer flags

❌ Don't use atomics when:

- Complex operations on multiple variables
- Need to maintain invariants across variables
- Operations cannot be made atomic
- Readability is more important than performance
- You're not sure about memory ordering

Performance Comparison:

```
#include <atomic>
#include <mutex>
#include <chrono>

// Mutex version
std::mutex mtx;
int counter1 = 0;

void incrementMutex() {
    std::lock_guard<std::mutex> lock(mtx);
    counter1++;
}

// Atomic version
std::atomic<int> counter2{0};

void incrementAtomic() {
    counter2++;
}

// Atomic is typically 2-10x faster for simple operations
```

Summary:

Aspect	Mutex	Atomic
Performance	Slower (syscall overhead)	Faster (hardware support)
Use case	Complex critical sections	Simple operations
Complexity	Easy to understand	Requires memory ordering knowledge
Composability	Can protect multiple operations	Single operations only
Deadlock risk	Yes	No

Rule of thumb: Use atomics for simple operations on single variables. Use mutexes for complex operations or multiple variables.

8. Explain the memory ordering models in C++.

Answer:

C++ memory ordering controls how memory operations are ordered and synchronized across threads. It's crucial for lock-free programming and atomic operations.

Memory Ordering Types:

Memory Order	Guarantees	Use Case	Cost
`memory_order_relaxed`	No synchronization	Counters, statistics	Lowest
`memory_order_consume`	Data dependency (deprecated)	Rarely used	Low
`memory_order_acquire`	Acquire barrier	Reading shared data	Medium
`memory_order_release`	Release barrier	Publishing shared data	Medium
`memory_order_acq_rel`	Both acquire and release	Read-modify-write	Medium

Memory Order	Guarantees	Use Case	Cost
`memory_order_seq_cst`	Sequential consistency	Default, safest	Highest

1. `memory_order_relaxed` - No Ordering

```
#include <atomic>
#include <thread>
#include <iostream>

std::atomic<int> x{0}, y{0};
std::atomic<int> z{0};

void relaxedExample() {
    // Thread 1
    std::thread t1([&] {
        x.store(1, std::memory_order_relaxed);
        y.store(1, std::memory_order_relaxed);
    });

    // Thread 2
    std::thread t2([&] {
        // May see y=1 but x=0 (no ordering guarantee!)
        while (y.load(std::memory_order_relaxed) == 0) {}

        if (x.load(std::memory_order_relaxed) == 1) {
            ++z;
        }
    });

    t1.join();
    t2.join();

    // z may or may not be incremented!
    std::cout << "z = " << z << std::endl;
}

// Use for: Simple counters where exact ordering doesn't matter
std::atomic<long> hit_count{0};

void incrementHits() {
    hit_count.fetch_add(1, std::memory_order_relaxed);  // OK: order doesn't
matter
}
```

2. `memory_order_acquire` and `memory_order_release`

```cpp
#include <atomic>
#include <thread>
#include <cassert>

std::atomic<bool> ready{false};
int data = 0;

void acquireReleaseExample() {
    // Producer thread
    std::thread producer([&] {
        data = 42;  // Non-atomic write

        // Release: all previous writes are visible to threads that acquire
        ready.store(true, std::memory_order_release);
    });

    // Consumer thread
    std::thread consumer([&] {
        // Acquire: sees all writes done before the release
        while (!ready.load(std::memory_order_acquire)) {
            // Spin wait
        }

        assert(data == 42);  // Guaranteed!
    });

    producer.join();
    consumer.join();
}
```

3. Acquire-Release Synchronization Pattern

```cpp
#include <atomic>
#include <thread>
#include <vector>

class Message {
public:
    std::vector<int> data;
};

std::atomic<Message*> mailbox{nullptr};

void sender() {
    Message* msg = new Message();
    msg->data = {1, 2, 3, 4, 5};

    // Release: publishes the message and all its contents
    mailbox.store(msg, std::memory_order_release);
}

void receiver() {
```

```
    Message* msg = nullptr;

    // Acquire: receives the message and sees all its contents
    while (!(msg = mailbox.load(std::memory_order_acquire))) {
        std::this_thread::yield();
    }

    // Safe to access msg->data here
    for (int val : msg->data) {
        std::cout << val << " ";
    }

    delete msg;
}
```

4. `memory_order_acq_rel` - Read-Modify-Write

```
#include <atomic>
#include <thread>

std::atomic<int> sync{0};
int payload[10];

void acqRelExample() {
    // Thread 1
    std::thread t1([&] {
        for (int i = 0; i < 10; ++i) {
            payload[i] = i;
        }

        // Acquire-release: both publishes and synchronizes
        sync.fetch_add(1, std::memory_order_acq_rel);
    });

    // Thread 2
    std::thread t2([&] {
        // Wait for thread 1's update
        while (sync.load(std::memory_order_acquire) == 0) {}

        // Safe to read payload
        for (int i = 0; i < 10; ++i) {
            std::cout << payload[i] << " ";
        }
    });

    t1.join();
    t2.join();
}
```

5. `memory_order_seq_cst` - Sequential Consistency (Default)

```
#include <atomic>
#include <thread>

std::atomic<bool> x{false}, y{false};
std::atomic<int> z{0};

void seqCstExample() {
    // With seq_cst: total global ordering is guaranteed

    // Thread 1
    std::thread t1([&] {
        x.store(true, std::memory_order_seq_cst);
    });

    // Thread 2
    std::thread t2([&] {
        y.store(true, std::memory_order_seq_cst);
    });

    // Thread 3
    std::thread t3([&] {
        while (!x.load(std::memory_order_seq_cst)) {}

        if (y.load(std::memory_order_seq_cst)) {
            ++z;
        }
    });

    // Thread 4
    std::thread t4([&] {
        while (!y.load(std::memory_order_seq_cst)) {}

        if (x.load(std::memory_order_seq_cst)) {
            ++z;
        }
    });

    t1.join(); t2.join(); t3.join(); t4.join();

    // With seq_cst: z is guaranteed to be at least 1
    std::cout << "z = " << z << std::endl;
}
```

Real-World Example: Lock-Free Queue

```cpp
#include <atomic>
#include <memory>

template<typename T>
class LockFreeQueue {
private:
    struct Node {
        std::shared_ptr<T> data;
        std::atomic<Node*> next;

        Node() : next(nullptr) {}
    };

    std::atomic<Node*> head;
    std::atomic<Node*> tail;

public:
    LockFreeQueue() {
        Node* dummy = new Node();
        head.store(dummy, std::memory_order_relaxed);
        tail.store(dummy, std::memory_order_relaxed);
    }

    void push(T value) {
        auto data = std::make_shared<T>(std::move(value));
        Node* new_node = new Node();

        Node* old_tail = tail.load(std::memory_order_acquire);

        while (true) {
            Node* null_ptr = nullptr;

            if (old_tail->next.compare_exchange_strong(
                    null_ptr, new_node,
                    std::memory_order_release,    // Success
                    std::memory_order_acquire)) { // Failure

                old_tail->data = data;
                tail.store(new_node, std::memory_order_release);
                break;
            } else {
                old_tail = old_tail->next.load(std::memory_order_acquire);
            }
        }
    }

    std::shared_ptr<T> pop() {
        Node* old_head = head.load(std::memory_order_acquire);

        while (true) {
            Node* next = old_head->next.load(std::memory_order_acquire);

            if (next == nullptr) {
                return nullptr;  // Queue empty
            }

            if (head.compare_exchange_strong(
```

```
                    old_head, next,
                    std::memory_order_release,
                    std::memory_order_acquire)) {

                std::shared_ptr<T> result = next->data;
                delete old_head;
                return result;
            }
        }
    }
};
```

Example: Double-Checked Locking Pattern

```
#include <atomic>
#include <mutex>

class Singleton {
private:
    static std::atomic<Singleton*> instance;
    static std::mutex mtx;

    Singleton() = default;

public:
    static Singleton* getInstance() {
        // First check (no lock)
        Singleton* tmp = instance.load(std::memory_order_acquire);

        if (tmp == nullptr) {
            std::lock_guard<std::mutex> lock(mtx);

            // Second check (with lock)
            tmp = instance.load(std::memory_order_relaxed);
            if (tmp == nullptr) {
                tmp = new Singleton();
                instance.store(tmp, std::memory_order_release);
            }
        }

        return tmp;
    }
};

std::atomic<Singleton*> Singleton::instance{nullptr};
std::mutex Singleton::mtx;
```

Memory Ordering Decision Tree:

```
Need synchronization between threads?
├─ NO → Use memory_order_relaxed
│   (e.g., simple counters, statistics)
│
└─ YES
    ├─ Need total global ordering?
    │   └─ YES → Use memory_order_seq_cst (default)
    │       (safest, easiest, most expensive)
    │
    └─ NO (pairwise synchronization is enough)
        ├─ Publishing data (write)?
        │   └─ Use memory_order_release
        │
        ├─ Receiving data (read)?
        │   └─ Use memory_order_acquire
        │
        └─ Both (read-modify-write)?
            └─ Use memory_order_acq_rel
```

Performance Impact:

```cpp
// Benchmark results (approximate, architecture-dependent):
std::atomic<int> counter{0};

// Relaxed:  ~1-2 CPU cycles
counter.fetch_add(1, std::memory_order_relaxed);

// Acquire/Release: ~5-10 CPU cycles
counter.fetch_add(1, std::memory_order_acq_rel);

// Sequential: ~10-20 CPU cycles
counter.fetch_add(1, std::memory_order_seq_cst);
```

Common Pitfalls:

```cpp
// ❌ Wrong: Missing synchronization
std::atomic<bool> flag{false};
int data = 0;

void wrong() {
    data = 42;
    flag.store(true, std::memory_order_relaxed);  // Wrong! Data race!
}

// ✅ Correct: Use acquire-release
void correct() {
    data = 42;
```

```
    flag.store(true, std::memory_order_release);  // Correct!
}
```

Summary:

- **Relaxed**: No ordering, fastest, use for independent operations
- **Acquire**: Load synchronization, pairs with release
- **Release**: Store synchronization, pairs with acquire
- **Acq_rel**: Both, for read-modify-write operations
- **Seq_cst**: Total ordering, safest but slowest (default)

Default strategy: Start with `seq_cst`, optimize to acquire-release only if profiling shows it matters.

9. What is `std::condition_variable` and how do you use it?

Answer:

A `std::condition_variable` is a synchronization primitive that allows threads to wait for certain conditions to become true. It enables efficient thread coordination without busy-waiting.

Key Concepts:

- **Wait**: Thread blocks until notified
- **Notify**: Wake up waiting thread(s)
- **Predicate**: Condition to check (prevents spurious wakeups)
- **Must use with `std::unique_lock`**

Basic Example: Producer-Consumer

```
#include <condition_variable>
#include <mutex>
#include <queue>
#include <thread>
```

```
#include <iostream>

std::queue<int> buffer;
std::mutex mtx;
std::condition_variable cv;
const size_t MAX_BUFFER_SIZE = 10;
bool done = false;

void producer() {
    for (int i = 0; i < 20; ++i) {
        std::unique_lock<std::mutex> lock(mtx);

        // Wait if buffer is full
        cv.wait(lock, [] {
            return buffer.size() < MAX_BUFFER_SIZE || done;
        });

        buffer.push(i);
        std::cout << "Produced: " << i << " (size: " << buffer.size() << ")"
<< std::endl;

        lock.unlock();
        cv.notify_one();  // Notify consumer

        std::this_thread::sleep_for(std::chrono::milliseconds(50));
    }

    {
        std::lock_guard<std::mutex> lock(mtx);
        done = true;
    }
    cv.notify_all();
}

void consumer() {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx);

        // Wait until buffer has data or production is done
        cv.wait(lock, [] {
            return !buffer.empty() || done;
        });

        while (!buffer.empty()) {
            int value = buffer.front();
            buffer.pop();
            std::cout << "Consumed: " << value << " (size: " << buffer.size()
<< ")" << std::endl;
        }

        if (done && buffer.empty()) {
            break;
        }

        lock.unlock();
        cv.notify_one();  // Notify producer

        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
```

```
}

int main() {
    std::thread prod(producer);
    std::thread cons(consumer);

    prod.join();
    cons.join();

    return 0;
}
```

Why Predicates Are Important (Spurious Wakeups)

```
#include <condition_variable>
#include <mutex>
#include <thread>

std::mutex mtx;
std::condition_variable cv;
bool ready = false;

// ❌ Wrong: No predicate (vulnerable to spurious wakeup)
void wrongWait() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock);  // May wake up even if not notified!

    // ready might still be false!
    if (ready) {  // Need to check manually
        // Do work
    }
}

// ✅ Correct: With predicate
void correctWait() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [] { return ready; });  // Only proceeds when ready is true

    // Guaranteed: ready is true here
}

// ✅ Alternative: Manual loop
void manualLoopWait() {
    std::unique_lock<std::mutex> lock(mtx);
    while (!ready) {  // Loop handles spurious wakeups
        cv.wait(lock);
    }

    // Guaranteed: ready is true here
}
```

Thread-Safe Queue with Condition Variable

```
#include <condition_variable>
#include <mutex>
#include <queue>
#include <optional>

template<typename T>
class ThreadSafeQueue {
private:
    mutable std::mutex mtx;
    std::condition_variable cv;
    std::queue<T> queue;
    bool closed = false;

public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            if (closed) {
                throw std::runtime_error("Queue is closed");
            }
            queue.push(std::move(value));
        }
        cv.notify_one();
    }

    // Blocking pop
    std::optional<T> pop() {
        std::unique_lock<std::mutex> lock(mtx);

        cv.wait(lock, [this] {
            return !queue.empty() || closed;
        });

        if (queue.empty()) {
            return std::nullopt;  // Queue closed and empty
        }

        T value = std::move(queue.front());
        queue.pop();
        return value;
    }

    // Try pop with timeout
    std::optional<T> pop_for(std::chrono::milliseconds timeout) {
        std::unique_lock<std::mutex> lock(mtx);

        if (!cv.wait_for(lock, timeout, [this] {
            return !queue.empty() || closed;
        })) {
            return std::nullopt;  // Timeout
        }

        if (queue.empty()) {
```

```
            return std::nullopt;  // Closed
        }

        T value = std::move(queue.front());
        queue.pop();
        return value;
    }

    void close() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            closed = true;
        }
        cv.notify_all();
    }

    bool empty() const {
        std::lock_guard<std::mutex> lock(mtx);
        return queue.empty();
    }
};

// Usage
void queueExample() {
    ThreadSafeQueue<int> queue;

    std::thread producer([&] {
        for (int i = 0; i < 10; ++i) {
            queue.push(i);
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
        queue.close();
    });

    std::thread consumer([&] {
        while (auto value = queue.pop()) {
            std::cout << "Got: " << *value << std::endl;
        }
        std::cout << "Queue closed" << std::endl;
    });

    producer.join();
    consumer.join();
}
```

Multiple Producers, Multiple Consumers

```
#include <condition_variable>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>
```

```
std::queue<int> tasks;
std::mutex mtx;
std::condition_variable cv;
bool finished = false;

void producer(int id, int count) {
    for (int i = 0; i < count; ++i) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            tasks.push(id * 1000 + i);
        }
        cv.notify_one();
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}

void consumer(int id) {
    while (true) {
        std::unique_lock<std::mutex> lock(mtx);

        cv.wait(lock, [] {
            return !tasks.empty() || finished;
        });

        if (tasks.empty() && finished) {
            break;
        }

        if (!tasks.empty()) {
            int task = tasks.front();
            tasks.pop();
            lock.unlock();

            std::cout << "Consumer " << id << " processing task " << task <<
std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(50));
        }
    }
}

void multiProducerConsumer() {
    std::vector<std::thread> threads;

    // Start 3 producers
    for (int i = 0; i < 3; ++i) {
        threads.emplace_back(producer, i, 5);
    }

    // Start 2 consumers
    for (int i = 0; i < 2; ++i) {
        threads.emplace_back(consumer, i);
    }

    // Wait for producers
    for (size_t i = 0; i < 3; ++i) {
        threads[i].join();
    }

    // Signal consumers to finish
```

```
    {
        std::lock_guard<std::mutex> lock(mtx);
        finished = true;
    }
    cv.notify_all();

    // Wait for consumers
    for (size_t i = 3; i < threads.size(); ++i) {
        threads[i].join();
    }
}
```

Event Synchronization

```
#include <condition_variable>
#include <mutex>
#include <thread>

class Event {
    std::mutex mtx;
    std::condition_variable cv;
    bool signaled = false;

public:
    void wait() {
        std::unique_lock<std::mutex> lock(mtx);
        cv.wait(lock, [this] { return signaled; });
    }

    template<typename Rep, typename Period>
    bool wait_for(const std::chrono::duration<Rep, Period>& timeout) {
        std::unique_lock<std::mutex> lock(mtx);
        return cv.wait_for(lock, timeout, [this] { return signaled; });
    }

    void signal() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            signaled = true;
        }
        cv.notify_all();
    }

    void reset() {
        std::lock_guard<std::mutex> lock(mtx);
        signaled = false;
    }
};

// Usage
void eventExample() {
    Event start_event;
```

```
    std::thread worker([&] {
        std::cout << "Worker waiting for signal..." << std::endl;
        start_event.wait();
        std::cout << "Worker started!" << std::endl;
    });

    std::this_thread::sleep_for(std::chrono::seconds(1));
    std::cout << "Signaling worker..." << std::endl;
    start_event.signal();

    worker.join();
}
```

Barrier Implementation

```
#include <condition_variable>
#include <mutex>
#include <thread>
#include <vector>

class Barrier {
    std::mutex mtx;
    std::condition_variable cv;
    size_t threshold;
    size_t count;
    size_t generation;

public:
    explicit Barrier(size_t num_threads)
        : threshold(num_threads), count(num_threads), generation(0) {}

    void wait() {
        std::unique_lock<std::mutex> lock(mtx);
        size_t gen = generation;

        if (--count == 0) {
            // Last thread to arrive
            generation++;
            count = threshold;
            cv.notify_all();
        } else {
            // Wait for all threads
            cv.wait(lock, [this, gen] { return gen != generation; });
        }
    }
};

// Usage: Synchronize threads at checkpoints
void barrierExample() {
    const int num_threads = 5;
    Barrier barrier(num_threads);
    std::vector<std::thread> threads;
```

```
    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back([&barrier, i] {
            std::cout << "Thread " << i << " phase 1" << std::endl;
            barrier.wait();  // Synchronization point

            std::cout << "Thread " << i << " phase 2" << std::endl;
            barrier.wait();  // Another synchronization point

            std::cout << "Thread " << i << " done" << std::endl;
        });
    }

    for (auto& t : threads) {
        t.join();
    }
}
```

notify_one() vs notify_all()

```
std::condition_variable cv;
std::mutex mtx;
bool ready = false;

// notify_one(): Wakes up ONE waiting thread
void notifyOneExample() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }
    cv.notify_one();  // Only one thread wakes up
}

// notify_all(): Wakes up ALL waiting threads
void notifyAllExample() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }
    cv.notify_all();  // All waiting threads wake up
}

// Use notify_one() when: Only one thread should proceed
// Use notify_all() when: All threads should check the condition
```

Best Practices:

1. **Always use a predicate** to avoid spurious wakeups
2. **Use** `std::unique_lock` (not `std::lock_guard`)

3. **Keep critical sections short** after waking up
4. **Use `notify_one()`** when only one thread should proceed
5. **Use `notify_all()`** when multiple threads might need to proceed
6. **Protect the condition with the same mutex** used for waiting

Common Pitfalls:

```
// ❌ Wrong: Notify before unlock (inefficient)
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
    cv.notify_one();  // Thread wakes up but can't acquire lock yet
}

// ✅ Better: Unlock before notify
{
    std::lock_guard<std::mutex> lock(mtx);
    ready = true;
}
cv.notify_one();  // Thread can immediately acquire lock
```

10. What is the difference between `std::async` and `std::thread`?

Answer:

`std::async` and `std::thread` both enable concurrent execution, but they operate at different abstraction levels with different use cases.

Comparison Table:

Feature	`std::thread`	`std::async`
Abstraction level	Low-level	High-level
Return value	No	Yes (`std::future`)
Exception handling	Need try-catch in thread	Propagates via `future.get()`
Thread lifecycle	Manual join/detach	Automatic

Feature	std::thread	std::async
Launch policy	Always creates thread	Can defer or async
Resource management	Manual	Automatic
Complexity	More complex	Simpler

Example 1: Basic Usage

```
#include <thread>
#include <future>
#include <iostream>

int computeValue(int x) {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    return x * x;
}

// Using std::thread
void threadExample() {
    int result = 0;

    std::thread t([&result] {
        result = computeValue(5);
    });

    t.join();  // Must join or detach
    std::cout << "Result: " << result << std::endl;
}

// Using std::async
void asyncExample() {
    std::future<int> result = std::async(std::launch::async, computeValue, 5);

    // Do other work...

    std::cout << "Result: " << result.get() << std::endl;  // Automatic join
}
```

Example 2: Return Values

```
#include <thread>
#include <future>
#include <vector>

// std::thread: Need to capture result manually
void threadReturnExample() {
```

```
    std::vector<int> results(10);
    std::vector<std::thread> threads;

    for (int i = 0; i < 10; ++i) {
        threads.emplace_back([&results, i] {
            results[i] = i * i;  // Capture result manually
        });
    }

    for (auto& t : threads) {
        t.join();
    }

    for (int r : results) {
        std::cout << r << " ";
    }
}

// std::async: Automatic return via future
void asyncReturnExample() {
    std::vector<std::future<int>> futures;

    for (int i = 0; i < 10; ++i) {
        futures.push_back(std::async(std::launch::async, [i] {
            return i * i;  // Return naturally
        }));
    }

    for (auto& f : futures) {
        std::cout << f.get() << " ";  // Get result from future
    }
}
```

Example 3: Exception Handling

```
#include <thread>
#include <future>
#include <stdexcept>
#include <iostream>

int riskyOperation(int x) {
    if (x < 0) {
        throw std::runtime_error("Negative value!");
    }
    return x * x;
}

// std::thread: Exception kills the program
void threadExceptionBad() {
    std::thread t([] {
        riskyOperation(-5);  // Exception -> std::terminate() called!
    });
    t.join();
```

```
}

// std::thread: Need to catch in thread and propagate manually
void threadExceptionGood() {
    std::exception_ptr eptr;

    std::thread t([&eptr] {
        try {
            riskyOperation(-5);
        } catch (...) {
            eptr = std::current_exception();  // Capture exception
        }
    });

    t.join();

    if (eptr) {
        try {
            std::rethrow_exception(eptr);
        } catch (const std::exception& e) {
            std::cout << "Caught: " << e.what() << std::endl;
        }
    }
}

// std::async: Exception automatically propagated
void asyncException() {
    std::future<int> result = std::async(std::launch::async, riskyOperation,
-5);

    try {
        int value = result.get();  // Exception propagated here
    } catch (const std::exception& e) {
        std::cout << "Caught: " << e.what() << std::endl;
    }
}
```

Example 4: Launch Policies

```
#include <future>
#include <iostream>

int task() {
    std::cout << "Task executing on thread: "
              << std::this_thread::get_id() << std::endl;
    return 42;
}

void launchPolicyExample() {
    // async: Guaranteed to run on separate thread
    auto future1 = std::async(std::launch::async, task);

    // deferred: Runs on current thread when get() is called
```

```
    auto future2 = std::async(std::launch::deferred, task);

    // async | deferred: Implementation chooses
    auto future3 = std::async(std::launch::async | std::launch::deferred,
task);

    // Default: Same as async | deferred
    auto future4 = std::async(task);

    std::cout << "Main thread: " << std::this_thread::get_id() << std::endl;

    std::cout << "Getting future1..." << std::endl;
    future1.get();  // Already running

    std::cout << "Getting future2..." << std::endl;
    future2.get();  // Executes NOW on main thread

    future3.get();
    future4.get();
}
```

Example 5: Resource Management

```
#include <thread>
#include <future>
#include <vector>

// std::thread: Manual management
void threadResourceManagement() {
    std::vector<std::thread> threads;

    for (int i = 0; i < 10; ++i) {
        threads.emplace_back([i] {
            // Do work
        });
    }

    // Must explicitly join all threads
    for (auto& t : threads) {
        if (t.joinable()) {
            t.join();
        }
    }
}

// std::async: Automatic management
void asyncResourceManagement() {
    std::vector<std::future<void>> futures;

    for (int i = 0; i < 10; ++i) {
        futures.push_back(std::async(std::launch::async, [i] {
            // Do work
        }));
```

```
    }

    // Futures automatically wait in destructor
    // Or explicitly wait:
    for (auto& f : futures) {
        f.wait();
    }
}
```

Example 6: Parallel Algorithm Pattern

```
#include <future>
#include <vector>
#include <numeric>
#include <algorithm>

// Parallel sum using async
template<typename Iterator>
long long parallelSum(Iterator begin, Iterator end) {
    long long size = std::distance(begin, end);

    if (size < 10000) {
        return std::accumulate(begin, end, 0LL);
    }

    Iterator mid = begin;
    std::advance(mid, size / 2);

    // Launch async task for first half
    auto future = std::async(std::launch::async, parallelSum<Iterator>, begin,
mid);

    // Process second half on current thread
    long long second_half = parallelSum(mid, end);

    // Get result from async task
    return future.get() + second_half;
}

void parallelSumExample() {
    std::vector<int> data(1000000);
    std::iota(data.begin(), data.end(), 1);  // 1, 2, 3, ...

    long long result = parallelSum(data.begin(), data.end());
    std::cout << "Sum: " << result << std::endl;
}
```

Example 7: When to Use Each

```cpp
#include <thread>
#include <future>
#include <iostream>

// Use std::thread for: Long-running background tasks
void backgroundTaskExample() {
    std::thread worker([]{
        while (true) {
            // Process messages, handle events, etc.
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    });

    worker.detach();  // Run independently
}

// Use std::async for: Tasks that return a value
void computationalTaskExample() {
    auto future = std::async(std::launch::async, []{
        // Heavy computation
        return 42;
    });

    // Do other work...

    int result = future.get();  // Get result when needed
}

// Use std::thread for: Need precise control
void preciseControlExample() {
    std::thread t([] {
        // Set thread priority, affinity, etc.
    });

    // Set thread attributes
    // ...

    t.join();
}

// Use std::async for: Fire-and-forget with result
void fireAndForgetExample() {
    std::async(std::launch::async, [] {
        // Task that returns result
        return 42;
    });  // Future destructor waits for completion
}
```

Example 8: Thread Pool vs Async

```cpp
#include <future>
#include <vector>
```

```
// std::async may create many threads (resource intensive)
void tooManyThreads() {
    std::vector<std::future<int>> futures;

    // Creates 1000 threads! May exhaust resources
    for (int i = 0; i < 1000; ++i) {
        futures.push_back(std::async(std::launch::async, [i] {
            return i * i;
        }));
    }

    for (auto& f : futures) {
        f.get();
    }
}

// Better: Use deferred or thread pool for many tasks
void betterApproach() {
    std::vector<std::future<int>> futures;

    // Let implementation decide (may use thread pool)
    for (int i = 0; i < 1000; ++i) {
        futures.push_back(std::async(std::launch::deferred, [i] {
            return i * i;
        }));
    }

    for (auto& f : futures) {
        f.get();
    }
}
```

Future Status Checking

```
#include <future>
#include <iostream>

void futureStatusExample() {
    auto future = std::async(std::launch::async, [] {
        std::this_thread::sleep_for(std::chrono::seconds(2));
        return 42;
    });

    // Check status without blocking
    while (future.wait_for(std::chrono::milliseconds(100)) !=
std::future_status::ready) {
        std::cout << "Still waiting..." << std::endl;
    }

    std::cout << "Result: " << future.get() << std::endl;
}
```

When to Use Which:

Use `std::thread` when:

- ✅ Need precise thread control (priority, affinity)
- ✅ Long-running background tasks
- ✅ Need to detach threads
- ✅ Building thread pools or custom concurrency
- ✅ Don't need return values

Use `std::async` when:

- ✅ Need return values from tasks
- ✅ Want automatic exception propagation
- ✅ Simpler code is preferred
- ✅ Tasks are independent computations
- ✅ Want implementation to choose async vs deferred
- ✅ Quick parallel operations

Performance Considerations:

```
// std::thread: Lower overhead once created
// - Thread creation: ~50-100 microseconds
// - Good for: Long-running tasks

// std::async: May reuse threads (implementation-defined)
// - May have thread pool internally
// - Good for: Many short tasks
```

Summary:

- `std::thread`: Low-level, explicit control, manual management
- `std::async`: High-level, automatic management, returns `std::future`
- **Default choice**: Use `std::async` unless you need thread-specific features
- **Best practice**: Use the highest abstraction that meets your needs

11. Explain the producer-consumer problem and its solution.

Answer:

The **producer-consumer problem** is a classic synchronization problem where:

- **Producers** create data and add it to a shared buffer
- **Consumers** remove data from the buffer and process it
- Buffer has limited capacity
- Must handle empty and full conditions

Problem Challenges:

1. **Race conditions** on buffer access
2. **Buffer overflow** (producer adds to full buffer)
3. **Buffer underflow** (consumer reads from empty buffer)
4. **Multiple producers/consumers** coordination

Solution 1: Using Mutex and Condition Variables

```
#include <condition_variable>
#include <mutex>
#include <queue>
#include <thread>
#include <iostream>
#include <vector>

template<typename T>
class BoundedBuffer {
private:
    std::queue<T> buffer;
    const size_t capacity;

    std::mutex mtx;
    std::condition_variable not_full;   // Signals producer
    std::condition_variable not_empty;  // Signals consumer

    bool closed = false;

public:
```

```
    explicit BoundedBuffer(size_t cap) : capacity(cap) {}

    void produce(T item) {
        std::unique_lock<std::mutex> lock(mtx);

        // Wait while buffer is full
        not_full.wait(lock, [this] {
            return buffer.size() < capacity || closed;
        });

        if (closed) {
            throw std::runtime_error("Buffer closed");
        }

        buffer.push(std::move(item));
        std::cout << "Produced: " << buffer.back()
                  << " (size: " << buffer.size() << ")" << std::endl;

        lock.unlock();
        not_empty.notify_one();  // Wake up a consumer
    }

    std::optional<T> consume() {
        std::unique_lock<std::mutex> lock(mtx);

        // Wait while buffer is empty
        not_empty.wait(lock, [this] {
            return !buffer.empty() || closed;
        });

        if (buffer.empty()) {
            return std::nullopt;  // Buffer closed and empty
        }

        T item = std::move(buffer.front());
        buffer.pop();
        std::cout << "Consumed: " << item
                  << " (size: " << buffer.size() << ")" << std::endl;

        lock.unlock();
        not_full.notify_one();  // Wake up a producer

        return item;
    }

    void close() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            closed = true;
        }
        not_full.notify_all();
        not_empty.notify_all();
    }
};

// Multiple producers, multiple consumers
void producerConsumerExample() {
    BoundedBuffer<int> buffer(5);  // Capacity: 5
```

```
    // Create 2 producers
    std::vector<std::thread> producers;
    for (int i = 0; i < 2; ++i) {
        producers.emplace_back([&buffer, i] {
            try {
                for (int j = 0; j < 10; ++j) {
                    buffer.produce(i * 100 + j);

std::this_thread::sleep_for(std::chrono::milliseconds(50));
                }
            } catch (const std::exception& e) {
                std::cout << "Producer " << i << " stopped" << std::endl;
            }
        });
    }

    // Create 3 consumers
    std::vector<std::thread> consumers;
    for (int i = 0; i < 3; ++i) {
        consumers.emplace_back([&buffer, i] {
            while (auto item = buffer.consume()) {
                // Process item
                std::this_thread::sleep_for(std::chrono::milliseconds(100));
            }
            std::cout << "Consumer " << i << " finished" << std::endl;
        });
    }

    // Wait for producers to finish
    for (auto& t : producers) {
        t.join();
    }

    // Close buffer and wait for consumers
    buffer.close();
    for (auto& t : consumers) {
        t.join();
    }
}
```

Solution 2: Using Semaphores (C++20)

```
#include <semaphore>
#include <mutex>
#include <queue>
#include <thread>

template<typename T>
class SemaphoreBuffer {
private:
    std::queue<T> buffer;
    std::mutex mtx;
```

```
    std::counting_semaphore<> empty_slots;  // Counts empty slots
    std::counting_semaphore<> full_slots;   // Counts items

public:
    explicit SemaphoreBuffer(size_t capacity)
        : empty_slots(capacity), full_slots(0) {}

    void produce(T item) {
        empty_slots.acquire();  // Wait for empty slot

        {
            std::lock_guard<std::mutex> lock(mtx);
            buffer.push(std::move(item));
        }

        full_slots.release();  // Signal item available
    }

    T consume() {
        full_slots.acquire();  // Wait for item

        T item;
        {
            std::lock_guard<std::mutex> lock(mtx);
            item = std::move(buffer.front());
            buffer.pop();
        }

        empty_slots.release();  // Signal empty slot
        return item;
    }
};
```

Solution 3: Lock-Free Ring Buffer (Single Producer, Single Consumer)

```
#include <atomic>
#include <array>
#include <optional>

template<typename T, size_t Size>
class LockFreeRingBuffer {
private:
    std::array<T, Size> buffer;
    std::atomic<size_t> write_pos{0};
    std::atomic<size_t> read_pos{0};

    size_t next(size_t current) const {
        return (current + 1) % Size;
    }

public:
```

```
    bool produce(const T& item) {
        size_t current_write = write_pos.load(std::memory_order_relaxed);
        size_t next_write = next(current_write);

        // Buffer full?
        if (next_write == read_pos.load(std::memory_order_acquire)) {
            return false;
        }

        buffer[current_write] = item;
        write_pos.store(next_write, std::memory_order_release);
        return true;
    }

    std::optional<T> consume() {
        size_t current_read = read_pos.load(std::memory_order_relaxed);

        // Buffer empty?
        if (current_read == write_pos.load(std::memory_order_acquire)) {
            return std::nullopt;
        }

        T item = buffer[current_read];
        read_pos.store(next(current_read), std::memory_order_release);
        return item;
    }

    bool empty() const {
        return read_pos.load(std::memory_order_acquire) ==
               write_pos.load(std::memory_order_acquire);
    }

    bool full() const {
        size_t next_write = next(write_pos.load(std::memory_order_acquire));
        return next_write == read_pos.load(std::memory_order_acquire);
    }
};

// Usage: Single producer, single consumer (lock-free!)
void lockFreeExample() {
    LockFreeRingBuffer<int, 16> buffer;

    std::thread producer([&buffer] {
        for (int i = 0; i < 100; ++i) {
            while (!buffer.produce(i)) {
                // Buffer full, wait
                std::this_thread::yield();
            }
        }
    });

    std::thread consumer([&buffer] {
        int count = 0;
        while (count < 100) {
            if (auto item = buffer.consume()) {
                std::cout << "Consumed: " << *item << std::endl;
                count++;
            } else {
                // Buffer empty, wait
```

```
                std::this_thread::yield();
            }
        }
    });

    producer.join();
    consumer.join();
}
```

Solution 4: Using `std::queue` with Timeout

```
#include <condition_variable>
#include <mutex>
#include <queue>
#include <chrono>

template<typename T>
class TimedBuffer {
private:
    std::queue<T> buffer;
    size_t capacity;
    std::mutex mtx;
    std::condition_variable cv;

public:
    explicit TimedBuffer(size_t cap) : capacity(cap) {}

    bool produce(T item, std::chrono::milliseconds timeout) {
        std::unique_lock<std::mutex> lock(mtx);

        if (!cv.wait_for(lock, timeout, [this] {
            return buffer.size() < capacity;
        })) {
            return false;  // Timeout
        }

        buffer.push(std::move(item));
        cv.notify_one();
        return true;
    }

    std::optional<T> consume(std::chrono::milliseconds timeout) {
        std::unique_lock<std::mutex> lock(mtx);

        if (!cv.wait_for(lock, timeout, [this] {
            return !buffer.empty();
        })) {
            return std::nullopt;  // Timeout
        }

        T item = std::move(buffer.front());
        buffer.pop();
        cv.notify_one();
```

```
        return item;
    }
};
```

Real-World Example: Task Queue

```
#include <condition_variable>
#include <mutex>
#include <queue>
#include <thread>
#include <functional>
#include <vector>

class TaskQueue {
private:
    std::queue<std::function<void()>> tasks;
    std::mutex mtx;
    std::condition_variable cv;
    bool stopped = false;
    std::vector<std::thread> workers;

    void workerThread() {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(mtx);
                cv.wait(lock, [this] {
                    return stopped || !tasks.empty();
                });

                if (stopped && tasks.empty()) {
                    return;
                }

                task = std::move(tasks.front());
                tasks.pop();
            }

            task();  // Execute task
        }
    }

public:
    explicit TaskQueue(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back(&TaskQueue::workerThread, this);
        }
    }

    ~TaskQueue() {
        {
            std::lock_guard<std::mutex> lock(mtx);
```

```
            stopped = true;
        }
        cv.notify_all();

        for (auto& worker : workers) {
            worker.join();
        }
    }

    template<typename F>
    void submit(F&& task) {
        {
            std::lock_guard<std::mutex> lock(mtx);
            tasks.emplace(std::forward<F>(task));
        }
        cv.notify_one();
    }
};

// Usage
void taskQueueExample() {
    TaskQueue queue(4);  // 4 worker threads

    // Submit tasks
    for (int i = 0; i < 20; ++i) {
        queue.submit([i] {
            std::cout << "Task " << i << " executing on thread "
                      << std::this_thread::get_id() << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        });
    }

    // TaskQueue destructor waits for all tasks to complete
}
```

Key Design Decisions:

1. **Single vs Multiple Condition Variables**:
    - Single: Simpler, but may wake up wrong threads
    - Multiple: More efficient, wake specific thread types
2. **Capacity**:
    - Unbounded: May consume excessive memory
    - Bounded: Better resource control, backpressure
3. **Blocking vs Non-Blocking**:
    - Blocking: Simpler, may waste threads

    - Non-blocking: More complex, better performance

4. **Error Handling**:
    - Exceptions for errors
    - Optional for empty/timeout
    - Boolean for try operations

Performance Considerations:

- **Lock-based**: Good for most cases, ~1-10 microseconds latency
- **Lock-free**: Better for high-throughput, low-latency scenarios
- **Wait strategy**: Sleep, yield, spin, or hybrid

Summary:

The producer-consumer problem requires:

1. **Synchronization**: Mutex for thread safety
2. **Signaling**: Condition variables for coordination
3. **Capacity management**: Handle full/empty conditions
4. **Proper shutdown**: Clean termination mechanism

Common solutions:

- Mutex + condition variables (most common)
- Semaphores (C++20)
- Lock-free ring buffer (SPSC)
- Task queues with thread pools

12. What is a thread pool and why is it useful?

Answer:

A **thread pool** is a collection of pre-created worker threads that execute tasks from a shared queue. Instead of creating a new thread for each task, tasks are submitted to the pool and executed by available workers.

Benefits:

1. **Reduced overhead**: No thread creation/destruction per task
2. **Resource control**: Limit concurrent threads
3. **Better performance**: Thread reuse is faster
4. **Load balancing**: Distributes work across threads
5. **Improved scalability**: Handles many short tasks efficiently

Basic Thread Pool Implementation

```
#include <vector>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <future>

class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;

    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;

    void workerThread() {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(mtx);

                // Wait for task or stop signal
                cv.wait(lock, [this] {
                    return stop || !tasks.empty();
                });

                if (stop && tasks.empty()) {
                    return;
                }

                task = std::move(tasks.front());
                tasks.pop();
            }

            task();  // Execute task outside lock
        }
    }
```

```
public:
    explicit ThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back(&ThreadPool::workerThread, this);
        }
    }

    ~ThreadPool() {
        {
            std::unique_lock<std::mutex> lock(mtx);
            stop = true;
        }

        cv.notify_all();

        for (auto& worker : workers) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

    template<typename F>
    void submit(F&& task) {
        {
            std::unique_lock<std::mutex> lock(mtx);
            tasks.emplace(std::forward<F>(task));
        }
        cv.notify_one();
    }
};

// Usage
void basicThreadPoolExample() {
    ThreadPool pool(4);  // 4 worker threads

    for (int i = 0; i < 20; ++i) {
        pool.submit([i] {
            std::cout << "Task " << i << " on thread "
                      << std::this_thread::get_id() << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        });
    }

    // Pool destructor waits for all tasks
}
```

Advanced Thread Pool with Return Values

```
#include <vector>
#include <queue>
#include <thread>
```

```
#include <mutex>
#include <condition_variable>
#include <functional>
#include <future>
#include <memory>

class AdvancedThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;

    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;

    void workerThread() {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(mtx);
                cv.wait(lock, [this] {
                    return stop || !tasks.empty();
                });

                if (stop && tasks.empty()) {
                    return;
                }

                task = std::move(tasks.front());
                tasks.pop();
            }

            task();
        }
    }

public:
    explicit AdvancedThreadPool(size_t num_threads =
std::thread::hardware_concurrency()) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back(&AdvancedThreadPool::workerThread, this);
        }
    }

    ~AdvancedThreadPool() {
        {
            std::unique_lock<std::mutex> lock(mtx);
            stop = true;
        }

        cv.notify_all();

        for (auto& worker : workers) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }
```

```
    // Submit task with return value
    template<typename F, typename... Args>
    auto submit(F&& f, Args&&... args) -> std::future<decltype(f(args...))> {
        using return_type = decltype(f(args...));

        auto task = std::make_shared<std::packaged_task<return_type()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...)
        );

        std::future<return_type> result = task->get_future();

        {
            std::unique_lock<std::mutex> lock(mtx);

            if (stop) {
                throw std::runtime_error("Submit on stopped ThreadPool");
            }

            tasks.emplace([task]() { (*task)(); });
        }

        cv.notify_one();
        return result;
    }
};

// Usage with return values
void advancedThreadPoolExample() {
    AdvancedThreadPool pool(4);

    std::vector<std::future<int>> results;

    for (int i = 0; i < 10; ++i) {
        results.push_back(pool.submit([i] {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
            return i * i;
        }));
    }

    for (auto& result : results) {
        std::cout << "Result: " << result.get() << std::endl;
    }
}
```

Thread Pool with Priority Queue

```
#include <queue>
#include <functional>

class PriorityThreadPool {
private:
    struct Task {
```

```
        int priority;
        std::function<void()> func;

        bool operator<(const Task& other) const {
            return priority < other.priority;  // Higher priority first
        }
    };

    std::vector<std::thread> workers;
    std::priority_queue<Task> tasks;

    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;

    void workerThread() {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(mtx);
                cv.wait(lock, [this] {
                    return stop || !tasks.empty();
                });

                if (stop && tasks.empty()) {
                    return;
                }

                task = std::move(tasks.top().func);
                tasks.pop();
            }

            task();
        }
    }

public:
    explicit PriorityThreadPool(size_t num_threads) {
        for (size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back(&PriorityThreadPool::workerThread, this);
        }
    }

    ~PriorityThreadPool() {
        {
            std::unique_lock<std::mutex> lock(mtx);
            stop = true;
        }
        cv.notify_all();

        for (auto& worker : workers) {
            worker.join();
        }
    }

    template<typename F>
    void submit(F&& task, int priority = 0) {
        {
```

```
            std::unique_lock<std::mutex> lock(mtx);
            tasks.push({priority, std::forward<F>(task)});
        }
        cv.notify_one();
    }
};

// Usage
void priorityPoolExample() {
    PriorityThreadPool pool(2);

    pool.submit([] { std::cout << "Low priority" << std::endl; }, 1);
    pool.submit([] { std::cout << "High priority" << std::endl; }, 10);
    pool.submit([] { std::cout << "Medium priority" << std::endl; }, 5);

    std::this_thread::sleep_for(std::chrono::seconds(1));
}
```

Dynamic Thread Pool (Scales Up/Down)

```
class DynamicThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;

    std::mutex mtx;
    std::condition_variable cv;
    bool stop = false;

    size_t min_threads;
    size_t max_threads;
    std::atomic<size_t> active_threads{0};

    void workerThread() {
        while (true) {
            std::function<void()> task;

            {
                std::unique_lock<std::mutex> lock(mtx);

                // Wait with timeout
                if (!cv.wait_for(lock, std::chrono::seconds(60), [this] {
                    return stop || !tasks.empty();
                })) {
                    // Timeout: shrink pool if above minimum
                    if (workers.size() > min_threads) {
                        return;  // Exit thread
                    }
                    continue;
                }

                if (stop && tasks.empty()) {
                    return;
```

```
                }

                task = std::move(tasks.front());
                tasks.pop();
            }

            active_threads++;
            task();
            active_threads--;
        }
    }

public:
    DynamicThreadPool(size_t min_threads, size_t max_threads)
        : min_threads(min_threads), max_threads(max_threads) {

        for (size_t i = 0; i < min_threads; ++i) {
            workers.emplace_back(&DynamicThreadPool::workerThread, this);
        }
    }

    ~DynamicThreadPool() {
        {
            std::unique_lock<std::mutex> lock(mtx);
            stop = true;
        }
        cv.notify_all();

        for (auto& worker : workers) {
            if (worker.joinable()) {
                worker.join();
            }
        }
    }

    template<typename F>
    void submit(F&& task) {
        {
            std::unique_lock<std::mutex> lock(mtx);
            tasks.emplace(std::forward<F>(task));

            // Scale up if needed
            if (tasks.size() > workers.size() && workers.size() < max_threads)
{
                workers.emplace_back(&DynamicThreadPool::workerThread, this);
            }
        }
        cv.notify_one();
    }
};
```

Real-World Example: Web Server Request Handler

```cpp
#include <string>
#include <iostream>

struct Request {
    int id;
    std::string data;
};

struct Response {
    int id;
    std::string result;
};

class RequestHandler {
private:
    AdvancedThreadPool pool;

    Response processRequest(const Request& req) {
        // Simulate processing
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        return {req.id, "Processed: " + req.data};
    }

public:
    RequestHandler() : pool(8) {}  // 8 worker threads

    std::future<Response> handleRequest(Request req) {
        return pool.submit([this, req] {
            return processRequest(req);
        });
    }
};

void webServerExample() {
    RequestHandler handler;
    std::vector<std::future<Response>> responses;

    // Handle 100 requests concurrently
    for (int i = 0; i < 100; ++i) {
        Request req{i, "Request data " + std::to_string(i)};
        responses.push_back(handler.handleRequest(req));
    }

    // Get all responses
    for (auto& future : responses) {
        Response resp = future.get();
        std::cout << "Response " << resp.id << ": " << resp.result <<
std::endl;
    }
}
```

Performance Comparison

```
#include <chrono>

void performanceComparison() {
    const int num_tasks = 1000;

    // Without thread pool (create thread per task)
    auto start1 = std::chrono::high_resolution_clock::now();
    {
        std::vector<std::thread> threads;
        for (int i = 0; i < num_tasks; ++i) {
            threads.emplace_back([] {
                std::this_thread::sleep_for(std::chrono::microseconds(100));
            });
        }
        for (auto& t : threads) t.join();
    }
    auto end1 = std::chrono::high_resolution_clock::now();

    // With thread pool
    auto start2 = std::chrono::high_resolution_clock::now();
    {
        AdvancedThreadPool pool(std::thread::hardware_concurrency());
        std::vector<std::future<void>> futures;

        for (int i = 0; i < num_tasks; ++i) {
            futures.push_back(pool.submit([] {
                std::this_thread::sleep_for(std::chrono::microseconds(100));
            }));
        }

        for (auto& f : futures) f.wait();
    }
    auto end2 = std::chrono::high_resolution_clock::now();

    auto duration1 = std::chrono::duration_cast<std::chrono::milliseconds>
(end1 - start1);
    auto duration2 = std::chrono::duration_cast<std::chrono::milliseconds>
(end2 - start2);

    std::cout << "Without pool: " << duration1.count() << "ms" << std::endl;
    std::cout << "With pool: " << duration2.count() << "ms" << std::endl;
    std::cout << "Speedup: " << (double)duration1.count() / duration2.count()
<< "x" << std::endl;
}
```

When to Use Thread Pools:

✅ Use thread pools when:

- Many short-lived tasks
- Need to control resource usage

- Want to avoid thread creation overhead
- Tasks are independent and parallelizable
- Building server applications
- Batch processing

❌ **Don't use thread pools when:**

- Few long-running tasks
- Tasks have complex dependencies
- Need specific thread configuration per task
- Task lifetime matches application lifetime

Key Design Considerations:

1. **Pool size**:
    - CPU-bound: `std::thread::hardware_concurrency()`
    - I/O-bound: 2-4x hardware threads
2. **Queue size**:
    - Bounded: Backpressure, prevents memory exhaustion
    - Unbounded: No blocking, but memory risk
3. **Shutdown**:
    - Graceful: Finish pending tasks
    - Immediate: Cancel pending tasks
4. **Error handling**:
    - Propagate exceptions via futures
    - Log errors in tasks

Summary:

Thread pools are essential for:

- **Performance**: Reuse threads instead of creating new ones
- **Scalability**: Handle many tasks with fixed thread count
- **Resource management**: Control concurrent execution

- **Simplicity**: Abstract away thread management

Rule of thumb: Use thread pools for most concurrent applications unless you need specific thread control.

13. What are the different types of mutexes in C++?

Answer:

C++ provides several mutex types for different synchronization needs:

1. `std::mutex` - Basic Mutex

Standard mutual exclusion primitive.

```
#include <mutex>
#include <thread>

std::mutex mtx;
int shared_data = 0;

void basicMutexExample() {
    std::lock_guard<std::mutex> lock(mtx);
    shared_data++;  // Thread-safe
}
```

Characteristics:

- Non-recursive (cannot relock from same thread)
- Non-timed (blocking only)
- Most commonly used
- Lowest overhead

2. `std::recursive_mutex` - Recursive Mutex

Allows same thread to lock multiple times.

```
#include <mutex>

class RecursiveExample {
    std::recursive_mutex mtx;
    int data = 0;

public:
    void increment() {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        data++;
    }

    void incrementTwice() {
        std::lock_guard<std::recursive_mutex> lock(mtx);
        increment();  // Locks again - OK with recursive_mutex
        increment();
    }
};
```

Characteristics:

- Can be locked multiple times by same thread
- Each lock must be matched with unlock
- Higher overhead than `std::mutex`
- Use sparingly (often indicates design issue)

3. `std::timed_mutex` - Mutex with Timeout

Supports timeout-based locking.

```
#include <mutex>
#include <chrono>

std::timed_mutex tmtx;

void timedMutexExample() {
    // Try to lock for up to 100ms
    if (tmtx.try_lock_for(std::chrono::milliseconds(100))) {
        // Got the lock
        // Do work
        tmtx.unlock();
    } else {
        // Timeout - couldn't get lock
        std::cout << "Timeout!" << std::endl;
    }

    // Or try until specific time point
    auto deadline = std::chrono::steady_clock::now() +
std::chrono::seconds(1);
```

```
    if (tmtx.try_lock_until(deadline)) {
        // Got the lock
        tmtx.unlock();
    }
}
```

Characteristics:

- `try_lock_for()`: Lock for duration
- `try_lock_until()`: Lock until time point
- Good for avoiding indefinite blocking
- Slightly higher overhead

4. `std::recursive_timed_mutex` - Recursive + Timed

Combines recursive and timed features.

```
#include <mutex>
#include <chrono>

std::recursive_timed_mutex rtmtx;

void recursiveTimedExample() {
    if (rtmtx.try_lock_for(std::chrono::milliseconds(100))) {
        // Can lock again from same thread
        if (rtmtx.try_lock_for(std::chrono::milliseconds(100))) {
            // Do nested work
            rtmtx.unlock();
        }
        rtmtx.unlock();
    }
}
```

Characteristics:

- Recursive + timeout capabilities
- Highest overhead
- Rarely needed in practice

5. `std::shared_mutex` (C++17) - Reader-Writer Lock

Allows multiple readers OR one writer.

```
#include <shared_mutex>
#include <thread>
#include <vector>

class ThreadSafeCache {
    mutable std::shared_mutex mtx;
    std::unordered_map<std::string, int> cache;

public:
    // Multiple readers can access simultaneously
    int get(const std::string& key) const {
        std::shared_lock<std::shared_mutex> lock(mtx);  // Shared/read lock
        auto it = cache.find(key);
        return (it != cache.end()) ? it->second : -1;
    }

    // Only one writer can access
    void set(const std::string& key, int value) {
        std::unique_lock<std::shared_mutex> lock(mtx);  // Exclusive/write
lock
        cache[key] = value;
    }
};

void sharedMutexExample() {
    ThreadSafeCache cache;

    // Multiple readers
    std::vector<std::thread> readers;
    for (int i = 0; i < 10; ++i) {
        readers.emplace_back([&cache] {
            for (int j = 0; j < 100; ++j) {
                int value = cache.get("key1");  // Concurrent reads OK
            }
        });
    }

    // Single writer
    std::thread writer([&cache] {
        for (int i = 0; i < 100; ++i) {
            cache.set("key1", i);  // Exclusive write
        }
    });

    for (auto& t : readers) t.join();
    writer.join();
}
```

Characteristics:

- Multiple readers: `shared_lock()` / `lock_shared()`
- Single writer: `unique_lock()` / `lock()`

- Optimizes read-heavy workloads
- More overhead than regular mutex

6. `std::shared_timed_mutex` (C++14) - Shared + Timed

Combines shared and timed features.

```
#include <shared_mutex>
#include <chrono>

std::shared_timed_mutex stmtx;

void sharedTimedExample() {
    // Try shared lock with timeout
    if (stmtx.try_lock_shared_for(std::chrono::milliseconds(100))) {
        // Read access
        stmtx.unlock_shared();
    }

    // Try exclusive lock with timeout
    if (stmtx.try_lock_for(std::chrono::milliseconds(100))) {
        // Write access
        stmtx.unlock();
    }
}
```

Characteristics:

- Combines shared and timed capabilities
- Most feature-rich
- Highest overhead

Comparison Table:

Mutex Type	Recursive	Timed	Shared	Overhead	Use Case
`std::mutex`	❌	❌	❌	Lowest	General purpose
`std::recursive_mutex`	✅	❌	❌	Medium	Recursive calls

Mutex Type	Recursive	Timed	Shared	Overhead	Use Case
`std::timed_mutex`	❌	✅	❌	Medium	Timeout needed
`std::recursive_timed_mutex`	✅	✅	❌	High	Both features
`std::shared_mutex`	❌	❌	✅	Medium	Read-heavy
`std::shared_timed_mutex`	❌	✅	✅	Highest	All features

Real-World Example: Configuration Manager

```
#include <shared_mutex>
#include <unordered_map>
#include <string>

class ConfigManager {
    mutable std::shared_mutex mtx;
    std::unordered_map<std::string, std::string> config;

public:
    // Frequent reads - use shared lock
    std::string get(const std::string& key) const {
        std::shared_lock<std::shared_mutex> lock(mtx);
        auto it = config.find(key);
        return (it != config.end()) ? it->second : "";
    }

    // Rare writes - use exclusive lock
    void set(const std::string& key, const std::string& value) {
        std::unique_lock<std::shared_mutex> lock(mtx);
        config[key] = value;
    }

    // Read multiple values atomically
    std::unordered_map<std::string, std::string> getAll() const {
        std::shared_lock<std::shared_mutex> lock(mtx);
        return config;
    }

    // Bulk update
    void updateAll(const std::unordered_map<std::string, std::string>&
newConfig) {
        std::unique_lock<std::shared_mutex> lock(mtx);
        config = newConfig;
```

```
    }
};
```

Decision Tree:

```
Need mutual exclusion?
│
├─ Need multiple readers, one writer?
│  └─ YES: std::shared_mutex
│      └─ Need timeout? → std::shared_timed_mutex
│
└─ NO (exclusive access only)
   │
   ├─ Need timeout?
   │  └─ YES: std::timed_mutex
   │      └─ Recursive? → std::recursive_timed_mutex
   │
   └─ NO (blocking only)
       └─ Recursive? → std::recursive_mutex
           └─ NO: std::mutex (default choice)
```

Performance Impact (Approximate):

```
// std::mutex: ~25 ns per lock/unlock
// std::recursive_mutex: ~35 ns per lock/unlock
// std::timed_mutex: ~30 ns per lock/unlock
// std::shared_mutex (read): ~40 ns per lock/unlock
// std::shared_mutex (write): ~50 ns per lock/unlock
```

Summary:

Default choice: `std::mutex` for most cases

Use specialized mutexes when:

- `recursive_mutex`: Recursive function calls (rare, often indicates design issue)
- `timed_mutex`: Need to avoid indefinite blocking
- `shared_mutex`: Read-heavy workloads with occasional writes
- Combined types: Need multiple features (rare)

14. Explain `std::shared_mutex` and reader-writer locks.

Answer:

`std::shared_mutex` (C++17) implements a **reader-writer lock** that allows:

- **Multiple readers** to access shared data simultaneously
- **Single writer** with exclusive access
- Optimizes scenarios with frequent reads and infrequent writes

Basic Concept:

```
Readers: Can share the lock (concurrent reads)
Writer:  Needs exclusive lock (blocks everyone)

State 1: [Reader1] [Reader2] [Reader3]  ← All reading simultaneously
State 2: [Writer]                        ← Exclusive write access
State 3: [Reader1] [Reader2]             ← Back to concurrent reads
```

Basic Example:

```cpp
#include <shared_mutex>
#include <map>
#include <thread>
#include <vector>

class PhoneBook {
private:
    mutable std::shared_mutex mtx;
    std::map<std::string, std::string> contacts;

public:
    // Read operation - shared lock (multiple threads can read)
    std::string lookup(const std::string& name) const {
        std::shared_lock<std::shared_mutex> lock(mtx);  // Shared/read lock

        auto it = contacts.find(name);
        return (it != contacts.end()) ? it->second : "Not found";
    }

    // Write operation - exclusive lock (only one thread can write)
    void add(const std::string& name, const std::string& phone) {
```

```
        std::unique_lock<std::shared_mutex> lock(mtx);  // Exclusive/write
lock

        contacts[name] = phone;
    }

    // Read all - shared lock
    size_t count() const {
        std::shared_lock<std::shared_mutex> lock(mtx);
        return contacts.size();
    }

    // Write operation
    void remove(const std::string& name) {
        std::unique_lock<std::shared_mutex> lock(mtx);
        contacts.erase(name);
    }
};

void readerWriterExample() {
    PhoneBook phonebook;

    // Writer thread
    std::thread writer([&] {
        for (int i = 0; i < 100; ++i) {
            phonebook.add("Person" + std::to_string(i), "555-" +
std::to_string(i));
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
        }
    });

    // Multiple reader threads (can run concurrently)
    std::vector<std::thread> readers;
    for (int i = 0; i < 10; ++i) {
        readers.emplace_back([&, i] {
            for (int j = 0; j < 100; ++j) {
                std::string result = phonebook.lookup("Person50");
                std::cout << "Reader " << i << ": " << result << std::endl;
            }
        });
    }

    writer.join();
    for (auto& t : readers) t.join();
}
```

Lock Types:

```
std::shared_mutex mtx;

// 1. Shared lock (read access) - multiple threads can hold simultaneously
std::shared_lock<std::shared_mutex> read_lock(mtx);
```

```
// 2. Unique lock (write access) - exclusive, blocks everyone
std::unique_lock<std::shared_mutex> write_lock(mtx);

// 3. Manual locking
mtx.lock_shared();    // Acquire shared
mtx.unlock_shared();  // Release shared

mtx.lock();           // Acquire exclusive
mtx.unlock();         // Release exclusive
```

Advanced Example: Caching System

```
#include <shared_mutex>
#include <unordered_map>
#include <optional>
#include <functional>

template<typename Key, typename Value>
class ThreadSafeCache {
private:
    mutable std::shared_mutex mtx;
    std::unordered_map<Key, Value> cache;
    std::function<Value(const Key&)> loader;

public:
    explicit ThreadSafeCache(std::function<Value(const Key&)> loader)
        : loader(std::move(loader)) {}

    // Get with read lock, upgrade to write lock if not found
    Value get(const Key& key) {
        // First try with shared lock (optimistic read)
        {
            std::shared_lock<std::shared_mutex> read_lock(mtx);
            auto it = cache.find(key);
            if (it != cache.end()) {
                return it->second;  // Cache hit - return immediately
            }
        }

        // Cache miss - need to load and update
        // Acquire exclusive lock
        std::unique_lock<std::shared_mutex> write_lock(mtx);

        // Double-check (another thread might have loaded it)
        auto it = cache.find(key);
        if (it != cache.end()) {
            return it->second;
        }

        // Load value
        Value value = loader(key);
        cache[key] = value;
```

```
        return value;
    }

    // Bulk read - shared lock
    std::vector<Value> getMultiple(const std::vector<Key>& keys) const {
        std::shared_lock<std::shared_mutex> lock(mtx);

        std::vector<Value> results;
        for (const auto& key : keys) {
            auto it = cache.find(key);
            if (it != cache.end()) {
                results.push_back(it->second);
            }
        }
        return results;
    }

    // Clear cache - exclusive lock
    void clear() {
        std::unique_lock<std::shared_mutex> lock(mtx);
        cache.clear();
    }

    // Get size - shared lock
    size_t size() const {
        std::shared_lock<std::shared_mutex> lock(mtx);
        return cache.size();
    }
};

// Usage
void cacheExample() {
    ThreadSafeCache<int, std::string> cache([](int key) {
        // Expensive operation
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        return "Value for " + std::to_string(key);
    });

    std::vector<std::thread> threads;

    // Multiple readers accessing same data
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back([&cache] {
            for (int j = 0; j < 100; ++j) {
                auto value = cache.get(42);  // Concurrent reads
            }
        });
    }

    for (auto& t : threads) t.join();
}
```

Upgrade Lock Pattern (Read → Write)

```
#include <shared_mutex>

class UpgradableData {
    mutable std::shared_mutex mtx;
    int data = 0;

public:
    void conditionalUpdate() {
        // Start with shared lock (read)
        std::shared_lock<std::shared_mutex> read_lock(mtx);

        if (data < 100) {
            // Need to modify - must upgrade to exclusive lock
            read_lock.unlock();  // Release shared lock

            std::unique_lock<std::shared_mutex> write_lock(mtx);  // Acquire
exclusive

            // Double-check (value might have changed)
            if (data < 100) {
                data++;
            }
        }
    }
};
```

Performance Comparison:

```
#include <mutex>
#include <shared_mutex>
#include <chrono>
#include <vector>

struct BenchmarkData {
    std::shared_mutex shared_mtx;
    std::mutex regular_mtx;
    int value = 0;
};

void benchmarkSharedMutex() {
    BenchmarkData data;
    const int num_readers = 10;
    const int num_iterations = 100000;

    // Benchmark with shared_mutex
    auto start1 = std::chrono::high_resolution_clock::now();
    {
        std::vector<std::thread> readers;
        for (int i = 0; i < num_readers; ++i) {
            readers.emplace_back([&data, num_iterations] {
                for (int j = 0; j < num_iterations; ++j) {
                    std::shared_lock<std::shared_mutex> lock(data.shared_mtx);
```

```
                    volatile int temp = data.value;  // Read
                }
            });
        }
        for (auto& t : readers) t.join();
    }
    auto end1 = std::chrono::high_resolution_clock::now();

    // Benchmark with regular mutex
    auto start2 = std::chrono::high_resolution_clock::now();
    {
        std::vector<std::thread> readers;
        for (int i = 0; i < num_readers; ++i) {
            readers.emplace_back([&data, num_iterations] {
                for (int j = 0; j < num_iterations; ++j) {
                    std::lock_guard<std::mutex> lock(data.regular_mtx);
                    volatile int temp = data.value;  // Read
                }
            });
        }
        for (auto& t : readers) t.join();
    }
    auto end2 = std::chrono::high_resolution_clock::now();

    auto duration1 = std::chrono::duration_cast<std::chrono::milliseconds>
(end1 - start1);
    auto duration2 = std::chrono::duration_cast<std::chrono::milliseconds>
(end2 - start2);

    std::cout << "shared_mutex: " << duration1.count() << "ms" << std::endl;
    std::cout << "regular mutex: " << duration2.count() << "ms" << std::endl;
    std::cout << "Speedup: " << (double)duration2.count() / duration1.count()
<< "x" << std::endl;
}
```

Read-Write Ratio Matters:

```
// High read/write ratio (90% reads): shared_mutex wins
// Read: 90%, Write: 10% → shared_mutex ~3-5x faster

// Balanced (50/50): shared_mutex similar or slower
// Read: 50%, Write: 50% → shared_mutex ~0.8-1.2x

// Write-heavy (10% reads): regular mutex wins
// Read: 10%, Write: 90% → regular mutex ~1.5x faster
```

Fairness and Starvation:

```
// Problem: Writer starvation
// Solution: Implementation-defined fairness policies

class FairReaderWriter {
    std::shared_mutex mtx;
    std::atomic<int> waiting_writers{0};

public:
    void read() {
        // Don't acquire if writers are waiting (fairness)
        while (waiting_writers.load() > 0) {
            std::this_thread::yield();
        }

        std::shared_lock<std::shared_mutex> lock(mtx);
        // Read data
    }

    void write() {
        waiting_writers++;
        std::unique_lock<std::shared_mutex> lock(mtx);
        waiting_writers--;
        // Write data
    }
};
```

Best Practices:

1. **Use for read-heavy workloads** (>70% reads)
2. **Keep critical sections short**
3. **Avoid upgrading locks** (deadlock risk)
4. **Measure performance** before committing
5. **Consider false sharing** with reader threads

When to Use:

✅ Use `std::shared_mutex` when:

- Frequent reads, infrequent writes (>70% reads)
- Read operations are expensive
- Multiple reader threads available
- Data structure rarely modified

❌ Use regular `std::mutex` when:

- Balanced read/write ratio
- Write-heavy workload
- Very short critical sections
- Single-threaded reads common

Summary:

`std::shared_mutex` enables:

- **Concurrent reads**: Multiple threads read simultaneously
- **Exclusive writes**: Single writer blocks all
- **Performance**: Up to 5x faster for read-heavy workloads
- **Trade-off**: Higher overhead than regular mutex

Key pattern: Optimize for reads, serialize writes

15. What is lock-free programming?

Answer: Programming without using locks, typically using atomic operations and memory ordering. Benefits:

- No deadlocks
- Better performance in some scenarios
- Progress guarantees
- Complexity: Much harder to implement correctly

16. What is the ABA problem in concurrent programming?

Answer: Occurs in lock-free algorithms when a value changes from A to B and back to A, making it appear unchanged. Solutions:

- Tagged pointers
- Hazard pointers
- Memory reclamation schemes
- Version numbers

17. Explain `std::call_once` and `std::once_flag`.

Answer: Ensures a function is called exactly once across all threads:

```
std::once_flag flag;
std::call_once(flag, initialize_function);
```

Thread-safe singleton initialization.

18. What is the difference between `join()` and `detach()` for threads?

Answer:

- `join()`: Blocks until thread completes, thread resources cleaned up
- `detach()`: Thread runs independently, resources cleaned up automatically
- Every thread must be either joined or detached before destruction

19. What are futures and promises in C++?

Answer:

- `std::promise`: Allows setting a value/exception from one thread
- `std::future`: Allows getting the value from another thread
- Provides a channel for one-time data transfer between threads

20. Explain thread-local storage.

Answer: `thread_local` storage class creates separate instances of variables for each thread. Each thread has its own copy, eliminating race conditions for that variable.

21. What is spurious wakeup in condition variables?

Answer: When `wait()` returns even though no `notify()` was called. Always use `wait()` in a loop or with a predicate:

```
cv.wait(lock, []{return condition;});
```

22. How do you implement a thread-safe singleton?

Answer: Several approaches:

- `std::call_once` with `std::once_flag`
- Double-checked locking with atomic
- Static local variable (C++11 guarantees thread safety)
- Eager initialization

23. What is the difference between cooperative and preemptive multitasking?

Answer:

- Cooperative: Threads voluntarily yield control (coroutines)
- Preemptive: OS forcibly switches threads (standard threading)
- C++20 introduces coroutines for cooperative multitasking

24. Explain memory barriers and their importance.

Answer: Memory barriers prevent certain types of memory reordering by the processor/compiler. Important for:

- Ensuring visibility of writes across threads
- Implementing lock-free algorithms
- Maintaining program order where needed

25. What are the common pitfalls in concurrent programming?

Answer:

- Race conditions
- Deadlocks
- Priority inversion
- Starvation
- False sharing
- Incorrect memory ordering
- Exception safety in multithreaded code
- Resource leaks
- Improper thread lifecycle management

Additional Tips for Interviews:

1. **Always mention thread safety** when discussing shared resources
2. **Know the performance implications** of different synchronization primitives
3. **Understand when to use which tool** (atomic vs mutex vs lock-free)
4. **Be familiar with common patterns** (producer-consumer, reader-writer, etc.)
5. **Practice implementing** basic concurrent data structures
6. **Understand the relationship** between hardware and software concurrency models

Code Examples to Practice:

- Thread-safe queue implementation
- Reader-writer lock implementation

- Thread pool implementation
- Lock-free stack
- Producer-consumer with condition variables
- Barrier synchronization primitive